

DevOps Lab Report

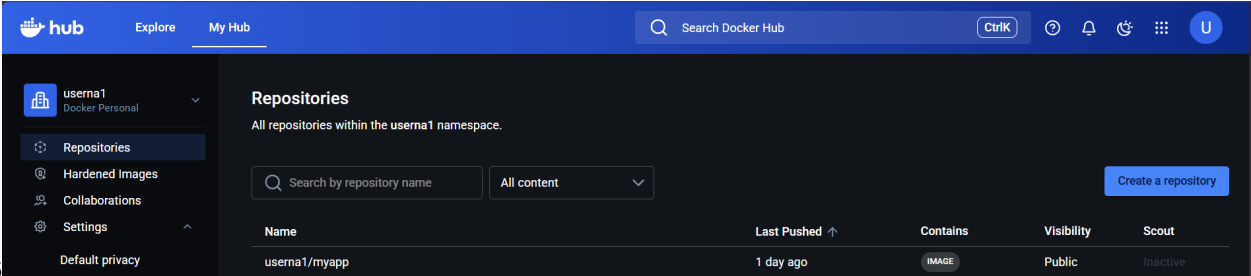
Tushar Mangal (500120513)

Contents

1	Containerization-and-DevOps	4
2	Containerization and DevOps	4
3	Name: Tushar Mangal	4
4	Sap Id: 500120513	4
4.1	About This Repository	4
4.2	Repository Structure	4
4.3	Theory Section	5
4.4	Lab Experiments	5
4.4.1	Lab 1 – Virtual Machines vs Containers	5
4.4.2	Lab 3	5
4.4.3	Lab 4	5
4.4.4	Lab 5	5
4.4.5	Lab 6	5
4.4.6	Lab 7	6
4.4.7	Lab 9	6
4.4.8	Lab 10	6
4.4.9	Lab 11	6
4.4.10	Lab 12	6
4.4.11	Assignment	6
5	Containerization-and-DevOps	6
6	Lab Experiment 1 – Comparison of Virtual Machines and Containers	6
6.1	Objective	6
6.2	Tools Used	6
7	Part A – Virtual Machine Setup	7
7.0.1	Step 1 – Download VirtualBox	7
7.0.2	Step 2 – Install Vagrant	7
7.0.3	Step 3 – Initialize Vagrant	7
7.0.4	Step 4 – Start VM	8
7.0.5	Step 5 – VM Created	8
7.0.6	Step 6 – VM Running	8
7.0.7	Step 7 – SSH into VM	8
7.0.8	Step 8 – Start Nginx in VM	9
7.0.9	Step 9 – Verify Nginx	9
7.0.10	Step 10 – Virtualization Enabled	10
8	Part B – Docker Container Setup	10
8.0.1	Step 1 – Open WSL	10

8.0.2	Step 2 – Run Docker Nginx	11
8.0.3	Step 3 – Verify Container Output	11
8.0.4	Step 4 – Monitor using htop	11
8.0.5	Step 5 – Check Memory	12
8.0.6	Step 6 – System Analyze	12
8.0.7	Step 7 – Docker Stats	12
8.1	Result	12
9	Containerization-and-DevOps	13
10	Experiment 4: Docker Essentials	13
10.1	Dockerfile .dockerignore tagging publishing	13
10.2	Part 1: Containerizing Applications with Dockerfile	13
10.2.1	Step 1: Create a Simple Application	13
10.2.2	Step 2: Create Dockerfile	14
10.3	Part 2: Using .dockerignore	16
10.3.1	Step 1: Create .dockerignore File	16
10.3.2	Step 2: Why .dockerignore is Important	18
10.4	Part 3: Building Docker Images	18
10.4.1	Step 1: Basic Build Command	18
10.4.2	Step 2: Tagging Images	18
10.4.3	Step 3: View Image Details	19
10.5	Part 4: Running Containers	20
10.5.1	Step 1: Run Container	20
10.5.2	Step 2: Manage Containers	21
10.6	Part 5: Multi-stage Builds	21
10.6.1	Step 1: Why Multi-stage Builds?	21
10.6.2	Step 2: Simple Multi-stage Dockerfile	22
10.6.3	Step 3: Build and Compare	24
10.7	Part 6: Publishing to Docker Hub	24
10.7.1	Step 1: Prepare for Publishing	24
10.7.2	Step 2: Pull and Run from Docker Hub	25
10.8	Part 7: Node.js Example (Quick Version)	25
10.8.1	Step 1: Node.js Application	25
10.8.2	Step 2: Node.js Dockerfile	27
10.8.3	Step 3: Build and Run	28
11	Containerization-and-DevOps	29
12	Lab 5: Docker Volumes, Environment Variables, Monitoring & Networks	29
12.1	Objective	29
12.2	Part 1: Docker Volumes – Persistent Data Storage	30
12.2.1	1. Anonymous & Named Volumes	30
12.2.2	2. Bind Mounts (Host Directory)	30
12.3	Part 2: Environment Variables	30
12.3.1	1. Building and Running a Flask App with ENV Vars	30
12.3.2	2. Testing Injected Environment Variables	33
12.4	Part 3: Docker Monitoring	33
12.4.1	Automated Monitoring Script	34
12.5	Part 4: Docker Networks	35
12.5.1	1. Creating a Network	35
12.5.2	2. Internal Network Communication	35
12.6	Part 5: Real-World Example (Multi-Container Setup)	36
12.6.1	1. Running Postgres and Redis on a Shared Network	36
12.6.2	2. Verifying the Services	37

12.7	Cleanup	37
12.8	Docker Commands Cheat Sheet	37
12.8.1	Volumes	37
12.8.2	Environment Variables	38
12.8.3	Monitoring	38
12.8.4	Networks	38
12.9	Key Takeaways	38
13	Containerization-and-DevOps	38
14	Experiment 6A: Comparison of Docker Run and Docker Compose	38
14.1	PART A – THEORY	38
14.1.1	1. Objective	38
14.1.2	2. Background Theory	38
14.1.3	3. Mapping: Docker Run vs Compose	39
14.1.4	4. Advantages of Docker Compose	39
14.2	PART B – PRACTICAL TASKS	39
14.2.1	Task 1: Single Container Comparison	39
14.2.2	Task 2: Multi-Container App (WordPress + MySQL)	41
14.3	PART C – CONVERSION TASKS (Theory)	42
14.3.1	Task 3: Convert Run → Compose	42
14.3.2	Task 4 & 5: Volumes, Networks, and Resource Limits	42
14.4	PART D – DOCKERFILE WITH COMPOSE	42
15	Experiment 6B: WordPress + MySQL using Compose	43
15.0.1	1. Run the Stack	44
15.0.2	2. Scaling	44
15.1	Docker Compose vs Swarm Comparison	45
15.2	Key Takeaways	45
16	Containerization-and-DevOps	45
17	Lab 7: CI/CD using Jenkins, GitHub & Docker Hub	45
17.1	1. Aim	45
17.2	2. Objective	45
17.3	3. What is CI/CD?	46
17.3.1	CI – Continuous Integration	46
17.3.2	CD – Continuous Deployment	46
17.3.3	Flow Diagram	46
17.4	4. What is Jenkins?	46
17.4.1	Key Features	46
17.5	5. Project Structure	46
17.6	6. Application Code (Flask)	46
17.7	7. Dockerfile	47
17.8	8. Jenkinsfile (Pipeline)	47
17.8.1	Pipeline Stages Explained	48
17.9	9. Jenkins Setup using Docker	48
17.9.1	Starting Jenkins	48
17.10	10. Jenkins Configuration	48
17.10.1	Step 1 – Add Docker Hub Credentials	48
17.10.2	Step 2 – Create a Pipeline Job	49
17.11	11. Webhook Integration	49
17.11.1	In GitHub	49
17.12	12. Execution Flow	49
17.13	13. Key Concept: withCredentials	49

17.13.1 Problem	49
17.13.2 Solution	49
17.14 14. Key Commands in Pipeline	50
17.15 15. Observations & Screenshots	50
17.15.1 Jenkins Running via Docker Compose	50
17.15.2 Jenkins Initial Setup & Dashboard	50
17.15.3 Adding Docker Hub Credentials in Jenkins	51
17.15.4 Pipeline Job Configuration	52
17.15.5 Pipeline Build Execution	52
17.15.6 Image Successfully Pushed to Docker Hub	53
17.16 	53
17.17 16. Result	53
17.18 Key Takeaways	53
18 Containerization-and-DevOps	53
18.1 Experiment 10: Working with SonarQube	53
19 Containerization-and-DevOps	61
19.1 Experiment 11: Orchestration via Docker Swarm	61
20 Containerization-and-DevOps	66
20.1 Experiment 12: Kubernetes	66

1 Containerization-and-DevOps

2 Containerization and DevOps

3 Name: Tushar Mangal

4 Sap Id: 500120513

4.1 About This Repository

This repository contains theory notes and lab experiments for the subject **Containerization and DevOps**. It focuses on understanding modern infrastructure concepts through both conceptual learning and hands-on practice.

Key areas covered: - Virtualization - Containerization - Docker - DevOps fundamentals

4.2 Repository Structure

```
Containerization-DevOps/
|-- Theory/
|   |-- Class1/
```

```
| |-- Class2/
| |-- Class3/
| |-- Class4/
| |-- Class5/
|-- Lab/
    |-- Lab-1/
    |-- Lab2/
    |-- Lab3/
    |-- Lab4/
    |-- Lab5/
    |-- Lab6/
    |-- Lab7/
```

4.3 Theory Section

This section contains class-wise theory notes.

Link to theory index:

- [Go to Theory README](#)

Direct links:

- [Go to Class 1 Notes](#)
 - [Go to Class 2 Notes](#)
 - [Go to Class 3 Notes](#)
 - [Go to Class 4 Notes](#)
 - [Go to Class 5 Notes](#)
-

4.4 Lab Experiments

4.4.1 Lab 1 – Virtual Machines vs Containers

This lab demonstrates the comparison between Virtual Machines and Containers using: - VirtualBox - Vagrant - Ubuntu - Docker - Nginx

Lab documentation: [Go to Lab 1 README](#)

4.4.2 Lab 3

Deploying NGINX Using Different Base Images and Comparing Image Layers

Lab documentation: [Go to Lab 3 Part1 README](#) Lab documentation: [Go to Lab 3 Part2 README](#)

4.4.3 Lab 4

Lab documentation: [Go to Lab 4 README](#)

4.4.4 Lab 5

Lab documentation: [Go to Lab 5 README](#)

4.4.5 Lab 6

Lab documentation: [Go to Lab 6 README](#)

4.4.6 Lab 7

Lab documentation: Go to Lab 7 README

4.4.7 Lab 9

Lab documentation: Go to Lab 9 README

4.4.8 Lab 10

Lab documentation: Go to Lab 10 README

4.4.9 Lab 11

Lab documentation: Go to Lab 11 README

4.4.10 Lab 12

Lab documentation: Go to Lab 12 README

4.4.11 Assignment

Assignment - 1: Assignment 1

5 Containerization-and-DevOps

6 Lab Experiment 1 – Comparison of Virtual Machines and Containers

6.1 Objective

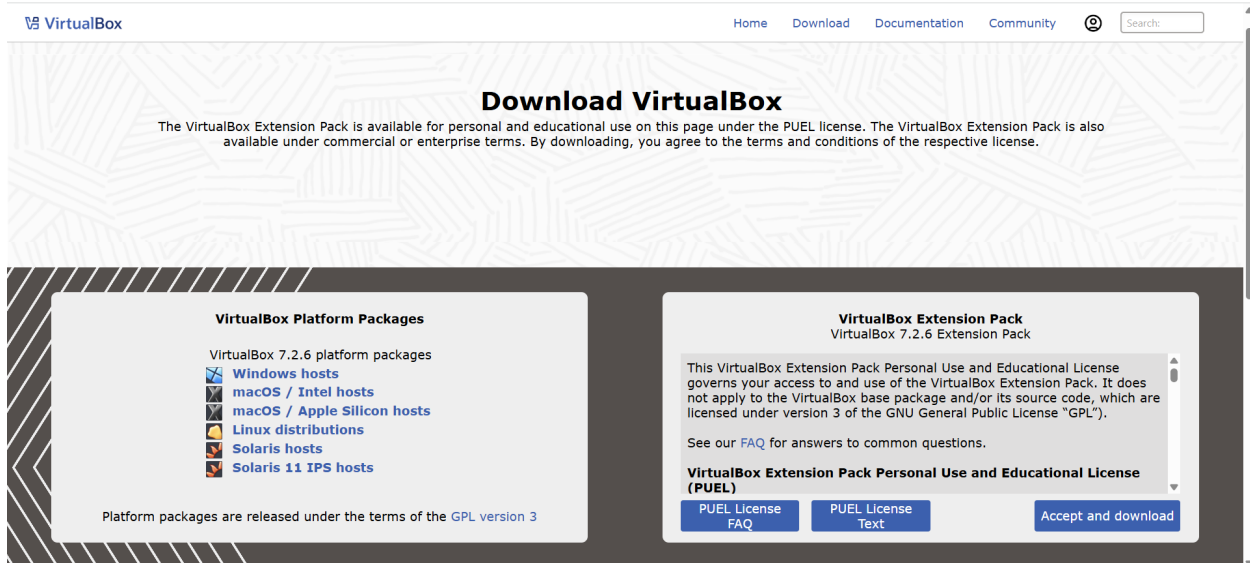
- Understand differences between Virtual Machines and Containers.
- Install and configure VM using VirtualBox and Vagrant.
- Install and configure Containers using Docker on WSL.
- Deploy Nginx web server in both environments.
- Observe system resource utilization.

6.2 Tools Used

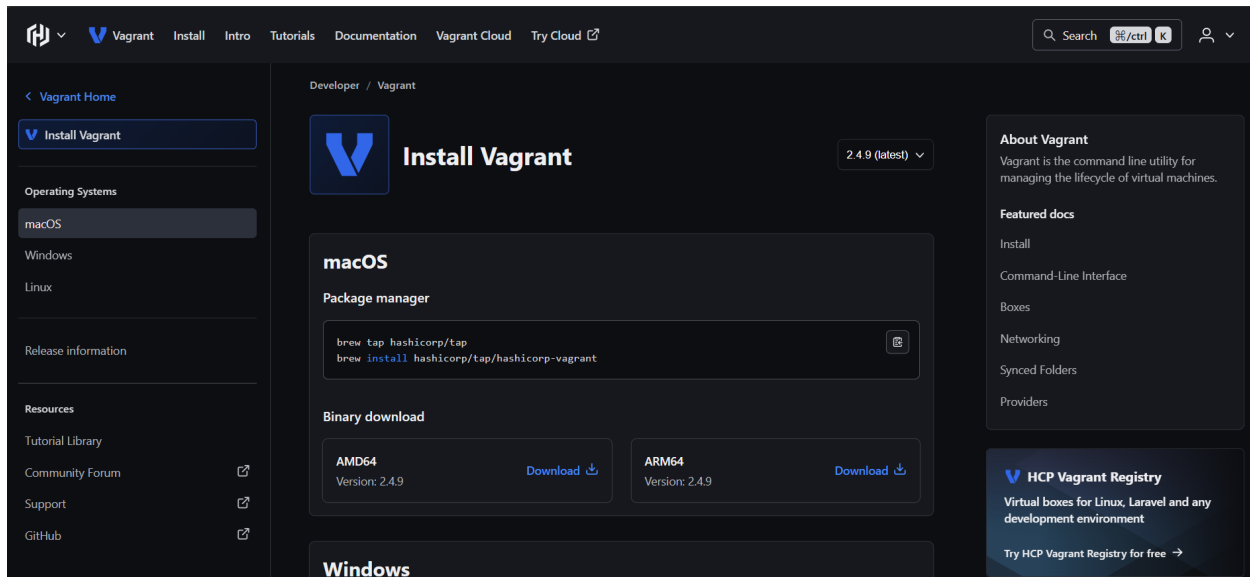
- VirtualBox
 - Vagrant
 - Ubuntu
 - WSL
 - Docker
 - Nginx
 - VS Code
-

7 Part A – Virtual Machine Setup

7.0.1 Step 1 – Download VirtualBox



7.0.2 Step 2 – Install Vagrant



7.0.3 Step 3 – Initialize Vagrant

vagrant init hashicorp/bionic64

```
PS D:\semseter 6\lab> vagrant init hashicorp/bionic64
A 'Vagrantfile' has been placed in this directory. You are now
ready to 'vagrant up' your first virtual environment! Please read
the comments in the Vagrantfile as well as documentation on
'vagrantup.com' for more information on using Vagrant.
PS D:\semseter 6\lab>
```

7.0.4 Step 4 – Start VM

vagrant up

```
PS D:\semseter 6\lab> vagrant init hashicorp/bionic64
A 'Vagrantfile' has been placed in this directory. You are now
ready to 'vagrant up' your first virtual environment! Please read
the comments in the Vagrantfile as well as documentation on
'vagrantup.com' for more information on using Vagrant.
PS D:\semseter 6\lab>
```

7.0.5 Step 5 – VM Created

```
==> default: Clearing any previously set network interfaces...
==> default: Preparing network interfaces based on configuration...
default: Adapter 1: nat
==> default: Forwarding ports...
default: 22 (guest) => 2222 (host) (adapter 1)
==> default: Booting VM...
==> default: Waiting for machine to boot. This may take a few minutes...
default: SSH address: 127.0.0.1:2222
default: SSH username: vagrant
default: SSH auth method: private key
default: Warning: Connection reset. Retrying...
default:
default: Vagrant insecure key detected. Vagrant will automatically replace
default: this with a newly generated keypair for better security.
default:
default: Inserting generated public key within guest...
default: Removing insecure key from the guest if it's present...
default: Key inserted! Disconnecting and reconnecting using new SSH key...
==> default: Machine booted and ready!
==> default: Checking for guest additions in VM...
default: The guest additions on this VM do not match the installed version of
default: VirtualBox! In most cases this is fine, but in rare cases it can
default: prevent things such as shared folders from working properly. If you see
default: shared folder errors, please make sure the guest additions within the
default: virtual machine match the version of VirtualBox you have installed on
default: your host and reload your VM.
default:
default: Guest Additions Version: 6.0.10
default: VirtualBox Version: 7.1
==> default: Mounting shared folders...
```

7.0.6 Step 6 – VM Running

```
PS D:\semseter 6\lab> vagrant ssh
Welcome to Ubuntu 18.04.3 LTS (GNU/Linux 4.15.0-58-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

System information as of Sat Jan 31 09:36:06 UTC 2026

System load:  0.5          Processes:           92
Usage of /:   2.5% of 61.80GB Users logged in:        0
Memory usage: 11%         IP address for eth0: 10.0.2.15
Swap usage:   0%

0 packages can be updated.
0 updates are security updates.

vagrant@vagrant:~$
```

7.0.7 Step 7 – SSH into VM

vagrant ssh

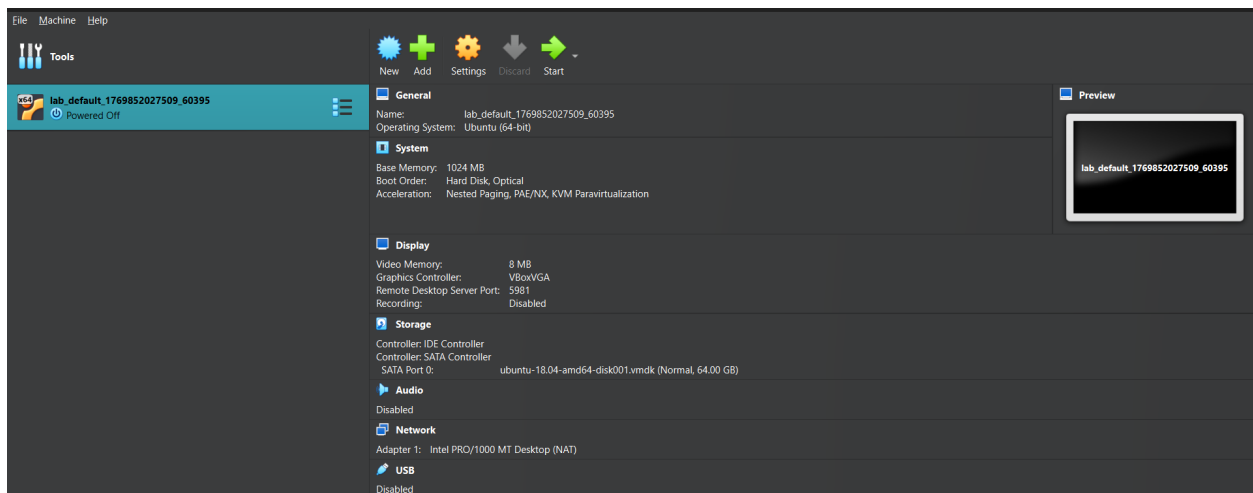

```
vagrant@vagrant:~$ sudo systemctl start nginx
vagrant@vagrant:~$ curl localhost
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
  body {
    width: 35em;
    margin: 0 auto;
    font-family: Tahoma, Verdana, Arial, sans-serif;
  }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
vagrant@vagrant:~$
```

7.0.8 Step 8 – Start Nginx in VM

```
sudo systemctl start nginx
```

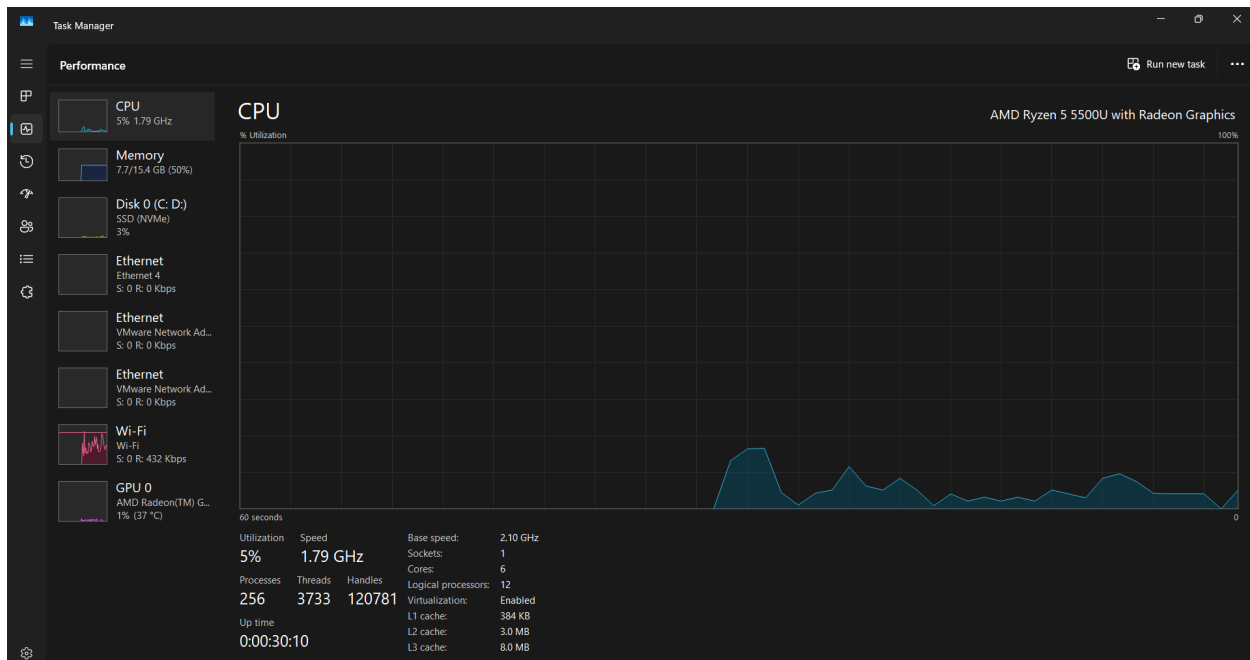


7.0.9 Step 9 – Verify Nginx

```
curl localhost
```



7.0.10 Step 10 – Virtualization Enabled



8 Part B – Docker Container Setup

8.0.1 Step 1 – Open WSL

```
PS D:\semseter 6\lab> wsl
tushar_mangal@LAPTOP-1DDBCFLP:/mnt/d/semseter 6/lab$ docker start ubuntu
Error response from daemon: No such container: ubuntu
Error: failed to start containers: ubuntu
```

8.0.2 Step 2 – Run Docker Nginx

```
docker run -d -p 8080:80 --name nginx-container nginx
```

```
tushar_mangal@LAPTOP-1DD8CFLP:/mnt/d/semseter 6/lab$ docker run -d -p 8080:80 --name nginx-container nginx
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
119d43eec815: Pull complete
700146c8ad64: Pull complete
d989100b8a84: Pull complete
500799c30424: Pull complete
10b68cfefee1: Pull complete
57f0dd1befee2: Pull complete
eaf8753feae0: Pull complete
Digest: sha256:c881927c4077710ac4b1da63b83aa163937fb47457950c267d92f7e4dedf4aec
Status: Downloaded newer image for nginx:latest
f67ef10ddcedbe2a775982a07aad6ee8568283bfc3df40031794a38676875442
```

8.0.3 Step 3 – Verify Container Output

```
curl localhost:8080
```

```
tushar_mangal@LAPTOP-1DD8CFLP:/mnt/d/semseter 6/lab$ curl localhost:8080
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```

8.0.4 Step 4 – Monitor using htop

```
htop
```

```

0[          0.0%] 3[          0.0%] 6[          0.0%] 9[          1.1%]
1[          0.7%] 4[          0.0%] 7[          3.3%] 10[         0.0%]
2[          0.0%] 5[          0.0%] 8[          0.0%] 11[         0.0%]
Mem[|||||]
Swp[
461M/7.45G Tasks: 43, 71 thr, 0 kthr: 1 running
0K/2.00G Load average: 0.26 0.12 0.04
Uptime: 00:05:02

Main I/O
PID USER PRI NI VIRT RES SHR S CPU% MEM% TIME+ Command
1259 tushar_man 20 0 6040 4736 3456 R 0.7 0.1 0:00.11 htop
305 root 20 0 2120M 50816 34944 S 0.4 0.7 0:00.18 /usr/bin/containerd
601 root 20 0 3136 1032 896 S 0.4 0.0 0:00.06 /init
1 root 20 0 21688 12396 9324 S 0.0 0.2 0:01.91 /sbin/init
2 root 20 0 3120 1920 1920 S 0.0 0.0 0:00.03 /init
7 root 20 0 3120 1792 1792 S 0.0 0.0 0:00.00 plan9 --control-socket 7 --log-level 4 --server-fd 8 --pipe-fd 10 --log-truncate
8 root 20 0 3120 1792 1792 S 0.0 0.0 0:00.00 plan9 --control-socket 7 --log-level 4 --server-fd 8 --pipe-fd 10 --log-truncate
9 root 20 0 3120 1920 1920 S 0.0 0.0 0:00.00 /init
46 root 19 -1 50352 16416 15648 S 0.0 0.2 0:00.50 /usr/lib/systemd/systemd-journald
98 root 20 0 25136 6144 4864 S 0.0 0.1 0:00.58 /usr/lib/systemd/systemd-udev
113 systemd-re 20 0 21588 12032 10112 S 0.0 0.2 0:00.33 /usr/lib/systemd/systemd-resolved
121 systemd-ti 20 0 91024 7424 6656 S 0.0 0.1 0:00.22 /usr/lib/systemd/systemd-timesyncd
169 systemd-ti 20 0 91024 7424 6656 S 0.0 0.1 0:00.00 /usr/lib/systemd/systemd-timesyncd
181 root 20 0 4236 2560 2304 S 0.0 0.0 0:00.03 /usr/sbin/cron -f -P
182 messagebus 20 0 9668 4864 4352 S 0.0 0.1 0:00.21 @dbus-daemon --system --address=systemd: --nofork --nopidfile --systemd-activation --sysl
195 root 20 0 17992 8448 7552 S 0.0 0.1 0:00.24 /usr/lib/systemd/systemd-logind
198 root 20 0 1714M 12160 10368 S 0.0 0.2 0:00.26 /usr/libexec/wsl-pro-service -vv
215 syslog 20 0 217M 5248 4224 S 0.0 0.1 0:00.11 /usr/sbin/rsyslogd -n -iNONE
227 root 20 0 3160 1920 1792 S 0.0 0.0 0:00.02 /sbin/agetty -o -p -- \u --noclear --keep-baud - 115200,38400,9600 vt220
231 root 20 0 2120M 50816 34944 S 0.0 0.7 0:00.36 /usr/bin/containerd
233 root 20 0 3116 1792 1664 S 0.0 0.0 0:00.02 /sbin/agetty -o -p -- \u --noclear - linux
239 root 20 0 104M 22144 12928 S 0.0 0.3 0:00.43 /usr/bin/python3 /usr/share/unattended-upgrades/unattended-upgrade-shutdown --wait-for-si
240 syslog 20 0 217M 5248 4224 S 0.0 0.1 0:00.03 /usr/sbin/rsyslogd -n -iNONE
241 syslog 20 0 217M 5248 4224 S 0.0 0.1 0:00.00 /usr/sbin/rsyslogd -n -iNONE
242 syslog 20 0 217M 5248 4224 S 0.0 0.1 0:00.01 /usr/sbin/rsyslogd -n -iNONE
243 root 20 0 1714M 12160 10368 S 0.0 0.2 0:00.02 /usr/libexec/wsl-pro-service -vv
244 root 20 0 1714M 12160 10368 S 0.0 0.2 0:00.00 /usr/libexec/wsl-pro-service -vv
245 root 20 0 1714M 12160 10368 S 0.0 0.2 0:00.00 /usr/libexec/wsl-pro-service -vv
246 root 20 0 1714M 12160 10368 S 0.0 0.2 0:00.01 /usr/libexec/wsl-pro-service -vv
248 root 20 0 1714M 12160 10368 S 0.0 0.2 0:00.00 /usr/libexec/wsl-pro-service -vv
F1Help F2Setup F3Search F4Filter F5Tree F6SortBy F7Nice F8Nice F9Kill F10Quit

```

8.0.5 Step 5 – Check Memory

free -h

```

tushar_mangal@LAPTOP-1DDBCF: /mnt/d/semseter 6/lab$ free -h
              total          used          free      shared  buff/cache   available
Mem:          7.5Gi          626Mi          6.0Gi          3.6Mi          1.0Gi          6.8Gi
Swap:         2.0Gi              0B          2.0Gi

```

8.0.6 Step 6 – System Analyze

systemd-analyze

```

tushar_mangal@LAPTOP-1DDBCF: /mnt/d/semseter 6/lab$ systemd-analyze
Startup finished in 5.856s (userspace)
graphical.target reached after 5.796s in userspace.

```

8.0.7 Step 7 – Docker Stats

docker stats

```

CONTAINER ID   NAME             CPU %       MEM USAGE / LIMIT   MEM %       NET I/O       BLOCK I/O    PIDS
f67ef10ddced   nginx-container  0.00%       10.43MiB / 7.45GiB  0.14%       1.72kB / 1.4kB   0B / 12.3kB  13

```

8.1 Result

Successfully compared VM and Container performance.

9 Containerization-and-DevOps

10 Experiment 4: Docker Essentials

10.1 Dockerfile .dockerignore tagging publishing

10.2 Part 1: Containerizing Applications with Dockerfile

10.2.1 Step 1: Create a Simple Application

Python Flask App:

```
mkdir my-flask-app
cd my-flask-app
```

```
Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows
PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-4> mkdir my-flask-app

Directory: C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-4

Mode                LastWriteTime         Length Name
----                -
d-----         21-02-2026         14:15         my-flask-app

PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-4> cd my-flask-app
```

app.py:

```
from flask import Flask
app = Flask(__name__)

@app.route('/')
def hello():
    return "Hello from Docker!"

@app.route('/health')
def health():
    return "OK"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

```
app.py X
app.py
1  from flask import Flask
2  app = Flask(__name__)
3
4  @app.route('/')
5  def hello():
6      return "Hello from Docker!"
7
8  @app.route('/health')
9  def health():
10     return "OK"
11
12 if __name__ == '__main__':
13     app.run(host='0.0.0.0', port=5000)
14
```

requirements.txt:

Flask==2.3.3

```
requirements.txt X
requirements.txt
1  Flask==2.3.3
2
```

10.2.2 Step 2: Create Dockerfile

Dockerfile:

```
# Use Python base image
FROM python:3.9-slim

# Set working directory
WORKDIR /app

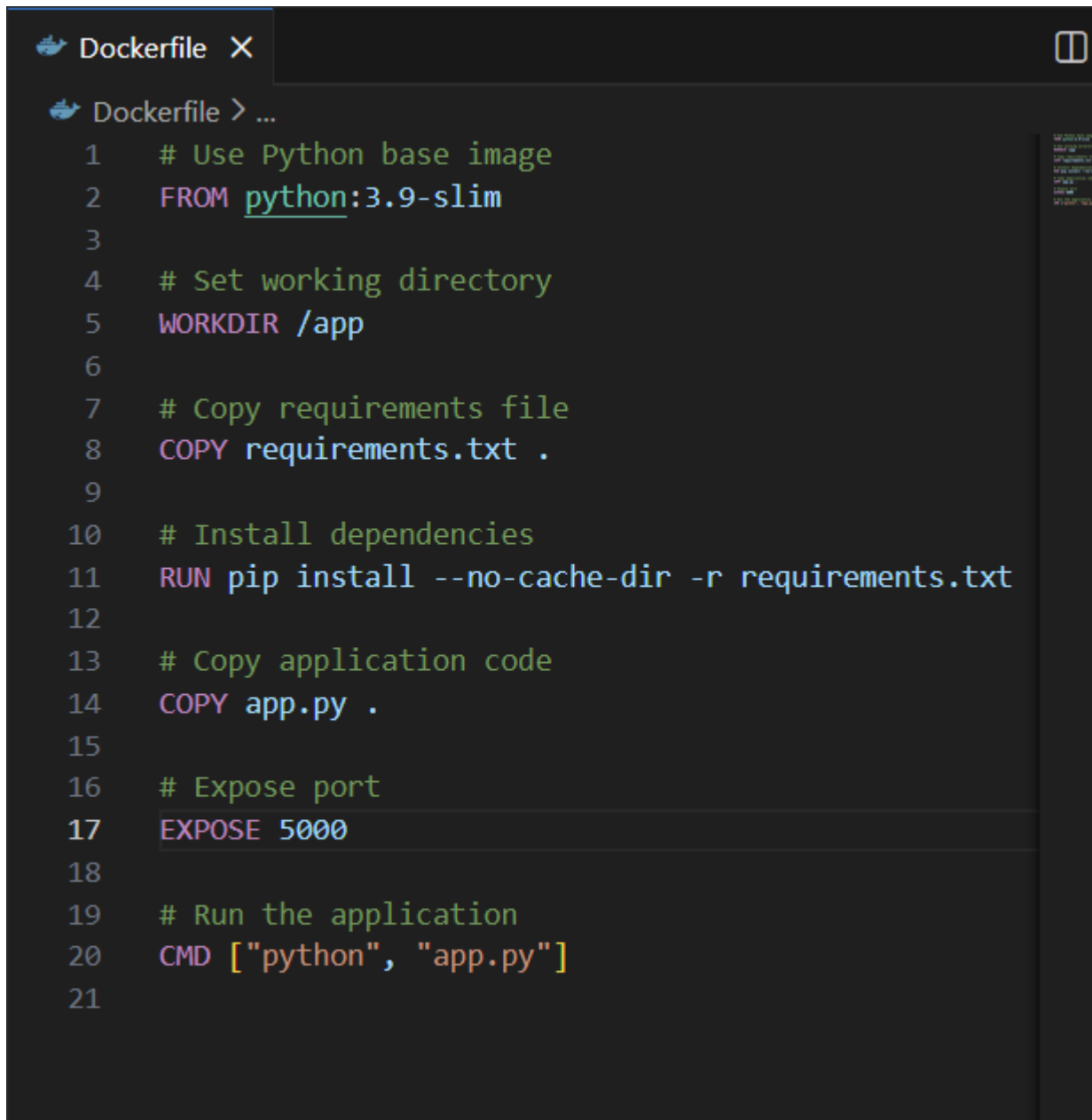
# Copy requirements file
COPY requirements.txt .

# Install dependencies
RUN pip install --no-cache-dir -r requirements.txt
```

```
# Copy application code
COPY app.py .

# Expose port
EXPOSE 5000

# Run the application
CMD ["python", "app.py"]
```

A screenshot of a Dockerfile editor interface. The top bar shows a Docker logo, the text 'Dockerfile', and a close button. Below the bar, the editor shows the Dockerfile content with line numbers from 1 to 21. The code is color-coded: comments are green, keywords like FROM, WORKDIR, COPY, RUN, EXPOSE, and CMD are pink, and file names or paths are blue. The code matches the text in the first block. On the right side, there is a vertical sidebar with a list of files or directories, including 'requirements.txt' and 'app.py'.

```
Dockerfile X
Dockerfile > ...
1  # Use Python base image
2  FROM python:3.9-slim
3
4  # Set working directory
5  WORKDIR /app
6
7  # Copy requirements file
8  COPY requirements.txt .
9
10 # Install dependencies
11 RUN pip install --no-cache-dir -r requirements.txt
12
13 # Copy application code
14 COPY app.py .
15
16 # Expose port
17 EXPOSE 5000
18
19 # Run the application
20 CMD ["python", "app.py"]
21
```

10.3 Part 2: Using .dockerignore

10.3.1 Step 1: Create .dockerignore File

```
.dockerignore:
# Python files
__pycache__/
*.pyc
*.pyo
*.pyd

# Environment files
.env
.venv
env/
venv/

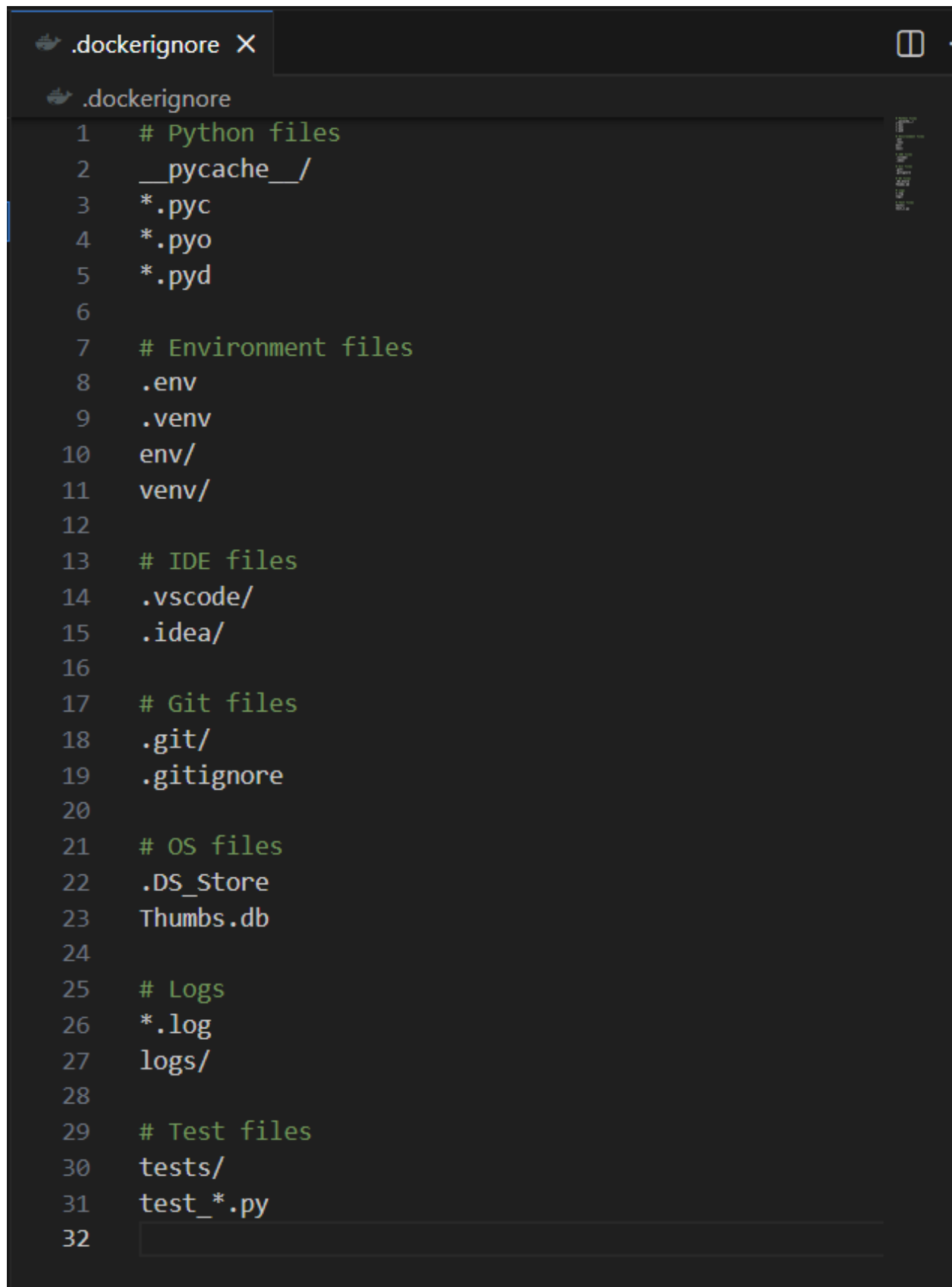
# IDE files
.vscode/
.idea/

# Git files
.git/
.gitignore

# OS files
.DS_Store
Thumbs.db

# Logs
*.log
logs/

# Test files
tests/
test_*.py
```


A screenshot of a code editor window showing a file named `.dockerignore`. The editor has a dark theme. The file content lists various file types and directories to be ignored by Docker. The lines are numbered from 1 to 32. The categories of files to be ignored are: Python files, Environment files, IDE files, Git files, OS files, Logs, and Test files.

```
1  # Python files
2  __pycache__/
3  *.pyc
4  *.pyo
5  *.pyd
6
7  # Environment files
8  .env
9  .venv
10 env/
11 venv/
12
13 # IDE files
14 .vscode/
15 .idea/
16
17 # Git files
18 .git/
19 .gitignore
20
21 # OS files
22 .DS_Store
23 Thumbs.db
24
25 # Logs
26 *.log
27 logs/
28
29 # Test files
30 tests/
31 test_*.py
32
```

10.3.2 Step 2: Why .dockerignore is Important

- Prevents unnecessary files from being copied
- Reduces image size
- Improves build speed
- Increases security

10.4 Part 3: Building Docker Images

10.4.1 Step 1: Basic Build Command

```
# Build image from Dockerfile
docker build -t my-flask-app .
```

```
# Check built images
docker images
```

```
View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/xwxm1kga08khgrgqtq5ctf2z3
PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-4\my-flask-app> docker build -t my-flask-app .
[+] Building 1.3s (10/10) FINISHED          docker:desktop-linux
=> [internal] load build definition from Dockerfile                                0.0s
=> => transferring dockerfile: 381B                                              0.0s
=> [internal] load metadata for docker.io/library/python:3.9-slim                0.6s
=> [internal] load .dockerignore                                                  0.0s
=> => transferring context: 304B                                                 0.0s
=> [1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16b             0.1s
=> => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16b           0.1s
=> [internal] load build context                                                 0.0s
=> => transferring context: 63B                                                  0.0s
=> CACHED [2/5] WORKDIR /app                                                      0.0s
=> CACHED [3/5] COPY requirements.txt .                                           0.0s
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt               0.0s
=> CACHED [5/5] COPY app.py .                                                    0.0s
=> exporting to image                                                            0.3s
=> => exporting layers                                                            0.0s
=> => exporting manifest sha256:c8d2616072f3cdd22c334d17834f3d624565          0.0s
=> => exporting config sha256:47bcef556a96a512e2e965ef7e584562a32fe1          0.0s
=> => exporting attestation manifest sha256:642430dc8c2f91124e563d74           0.1s
=> => exporting manifest list sha256:8300d23a75f1dc49dcb068b4992fff3          0.0s
=> => naming to docker.io/library/my-flask-app:latest                          0.0s
=> => unpacking to docker.io/library/my-flask-app:latest                        0.0s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/an34lbmcr6kgn7i3x5fb2g3ls
PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-4\my-flask-app> docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
cloud-robot:latest	1ae3bf81d146	857MB	191MB	U
my-flask-app:latest	8300d23a75f1	200MB	48.8MB	U
nginx-alpine:latest	3a47dbf1b178	16MB	4.74MB	U
nginx-ubuntu:latest	cb305251001b	199MB	52.7MB	U
nginx:latest	341bf0f3ce6c	240MB	65.8MB	U

10.4.2 Step 2: Tagging Images

```
# Tag with version number
docker build -t my-flask-app:1.0 .
```

```
# Tag with multiple tags
docker build -t my-flask-app:latest -t my-flask-app:1.0 .
```

```
# Tag with custom registry
docker build -t username/my-flask-app:1.0 .
```

```
# Tag existing image
docker tag my-flask-app:latest my-flask-app:v1.0
```

```

PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-4\my-flask-app> docker build -t my-flask-app:1.0 .
[+] Building 1.3s (10/10) FINISHED          docker:desktop-linux
=> [internal] load build definition from Dockerfile          0.0s
=> => transferring dockerfile: 381B                          0.0s
=> [internal] load metadata for docker.io/library/python:3.9-slim 0.6s
=> [internal] load .dockerignore                          0.0s
=> => transferring context: 304B                              0.0s
=> [1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16b 0.1s
=> => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16b 0.1s
=> [internal] load build context                          0.0s
=> => transferring context: 63B                              0.0s
=> CACHED [2/5] WORKDIR /app                              0.0s
=> CACHED [3/5] COPY requirements.txt .                    0.0s
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt 0.0s
=> CACHED [5/5] COPY app.py .                              0.0s
=> exporting to image                                    0.3s
=> => exporting layers                                      0.0s
=> => exporting manifest sha256:c8d2616072f3cdd22c334d17834f3d624565 0.0s
=> => exporting config sha256:47bcef556a96a512e2e965ef7e584562a32fe1 0.0s
=> => exporting attestation manifest sha256:bb49a0f6ee10a0a57978775 0.1s
=> => exporting manifest list sha256:c2771f6d4a602b095f232cc877ac064 0.0s
=> => naming to docker.io/library/my-flask-app:1.0          0.0s
=> => unpacking to docker.io/library/my-flask-app:1.0       0.0s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/sij6uo3tonwqjiw6kss2mmgm7
PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-4\my-flask-app> docker build -t my-flask-app:latest -t my-flask-app:1.0 .
[+] Building 1.2s (10/10) FINISHED          docker:desktop-linux
=> [internal] load build definition from Dockerfile          0.0s
=> => transferring dockerfile: 381B                          0.0s
=> [internal] load metadata for docker.io/library/python:3.9-slim 0.5s
=> [internal] load .dockerignore                          0.0s
=> => transferring context: 304B                              0.0s
=> [1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16b 0.1s
=> => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16b 0.1s
=> [internal] load build context                          0.0s
=> => transferring context: 63B                              0.0s
=> CACHED [2/5] WORKDIR /app                              0.0s
=> CACHED [3/5] COPY requirements.txt .                    0.0s
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt 0.0s
=> CACHED [5/5] COPY app.py .                              0.0s
=> exporting to image                                    0.3s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/f6tjka8hnalv38pjhf8q74am
PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-4\my-flask-app> docker build -t username/my-flask-app:1.0 .
[+] Building 1.3s (10/10) FINISHED          docker:desktop-linux
=> [internal] load build definition from Dockerfile          0.0s
=> => transferring dockerfile: 381B                          0.0s
=> [internal] load metadata for docker.io/library/python:3.9-slim 0.6s
=> [internal] load .dockerignore                          0.0s
=> => transferring context: 304B                              0.0s
=> [1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16b 0.1s
=> => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16b 0.1s
=> [internal] load build context                          0.0s
=> => transferring context: 63B                              0.0s
=> CACHED [2/5] WORKDIR /app                              0.0s
=> CACHED [3/5] COPY requirements.txt .                    0.0s
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt 0.0s
=> CACHED [5/5] COPY app.py .                              0.0s
=> exporting to image                                    0.3s
=> => exporting layers                                      0.0s
=> => exporting manifest sha256:c8d2616072f3cdd22c334d17834f3d624565 0.0s
=> => exporting config sha256:47bcef556a96a512e2e965ef7e584562a32fe1 0.0s
=> => exporting attestation manifest sha256:561e974dcddbf0ccf88f8be8 0.1s
=> => exporting manifest list sha256:678a8667bb828ccf8b6dca1c24b7d1a 0.0s
=> => naming to docker.io/username/my-flask-app:1.0          0.0s
=> => unpacking to docker.io/username/my-flask-app:1.0       0.0s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/y8xi8aica9wdly5pim1w8o0mu
PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-4\my-flask-app> docker tag my-flask-app:latest my-flask-app:v1.0

```

10.4.3 Step 3: View Image Details

List all images

docker images

Show image history

docker history my-flask-app

Inspect image details

docker inspect my-flask-app

```

PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-4\my-flask-app> docker images

```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
cldoud-robot:latest	1ae3bf81d146	857MB	191MB	
my-flask-app:1.0	67ab170e1c73	200MB	48.8MB	U
my-flask-app:latest	67ab170e1c73	200MB	48.8MB	
my-flask-app:v1.0	67ab170e1c73	200MB	48.8MB	
nginx-alpine:latest	3a47dbf1b178	16MB	4.74MB	U
nginx-ubuntu:latest	cb305251001b	199MB	52.7MB	U
nginx:latest	341bf0f3ce6c	240MB	65.8MB	U
username/my-flask-app:1.0	678a8667bb82	200MB	48.8MB	

```

PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-4\my-flask-app> docker history my-flask-app

```

IMAGE	CREATED	CREATED BY	SIZE	COMMENT
67ab170e1c73	About a minute ago	CMD ["python" "app.py"]	0B	buildkit.dockerfile.v0
<missing>	About a minute ago	EXPOSE [5000/tcp]	0B	buildkit.dockerfile.v0
<missing>	About a minute ago	COPY app.py . # buildkit	12.3kB	buildkit.dockerfile.v0
<missing>	About a minute ago	RUN /bin/sh -c pip install --no-cache-dir -r...	13.4MB	buildkit.dockerfile.v0
<missing>	About a minute ago	COPY requirements.txt . # buildkit	12.3kB	buildkit.dockerfile.v0
<missing>	28 hours ago	WORKDIR /app	8.19kB	buildkit.dockerfile.v0
<missing>	3 months ago	CMD ["python3"]	0B	buildkit.dockerfile.v0
<missing>	3 months ago	RUN /bin/sh -c set -eux; for src in idle3 p...	16.4kB	buildkit.dockerfile.v0
<missing>	3 months ago	RUN /bin/sh -c set -eux; savedAptMark="\$(a...	45.7MB	buildkit.dockerfile.v0
<missing>	3 months ago	ENV PYTHON_SHA256=00e07d7c0f2f0cc002432d1ee8...	0B	buildkit.dockerfile.v0
<missing>	3 months ago	ENV PYTHON_VERSION=3.9.25	0B	buildkit.dockerfile.v0
<missing>	3 months ago	ENV GPG_KEY=E3FF2839C048B25C084DEBE9B26995E3...	0B	buildkit.dockerfile.v0
<missing>	3 months ago	RUN /bin/sh -c set -eux; apt-get update; a...	4.94MB	buildkit.dockerfile.v0
<missing>	3 months ago	ENV LANG=C.UTF-8	0B	buildkit.dockerfile.v0
<missing>	3 months ago	ENV PATH=/usr/local/bin:/usr/local/sbin:/usr...	0B	buildkit.dockerfile.v0
<missing>	4 months ago	# debian.sh --arch 'amd64' out/ 'trixie' '@1...	87.4MB	debuerreotype 0.16

```

PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-4\my-flask-app> docker inspect my-flask-app

```

```

[
  {
    "Id": "sha256:67ab170e1c73b28dd6107a293781d17f87e045df9bf0683f1a5b4b74efd7d487",
    "RepoTags": [
      "my-flask-app:1.0",
      "my-flask-app:latest",
      "my-flask-app:v1.0"
    ],
    "RepoDigests": [
      "my-flask-app@sha256:67ab170e1c73b28dd6107a293781d17f87e045df9bf0683f1a5b4b74efd7d487"
    ],
  }
]

```

10.5 Part 4: Running Containers

10.5.1 Step 1: Run Container

```

# Run container with port mapping
docker run -d -p 5000:5000 --name flask-container my-flask-app

# Test the application
curl http://localhost:5000

# View running containers
docker ps

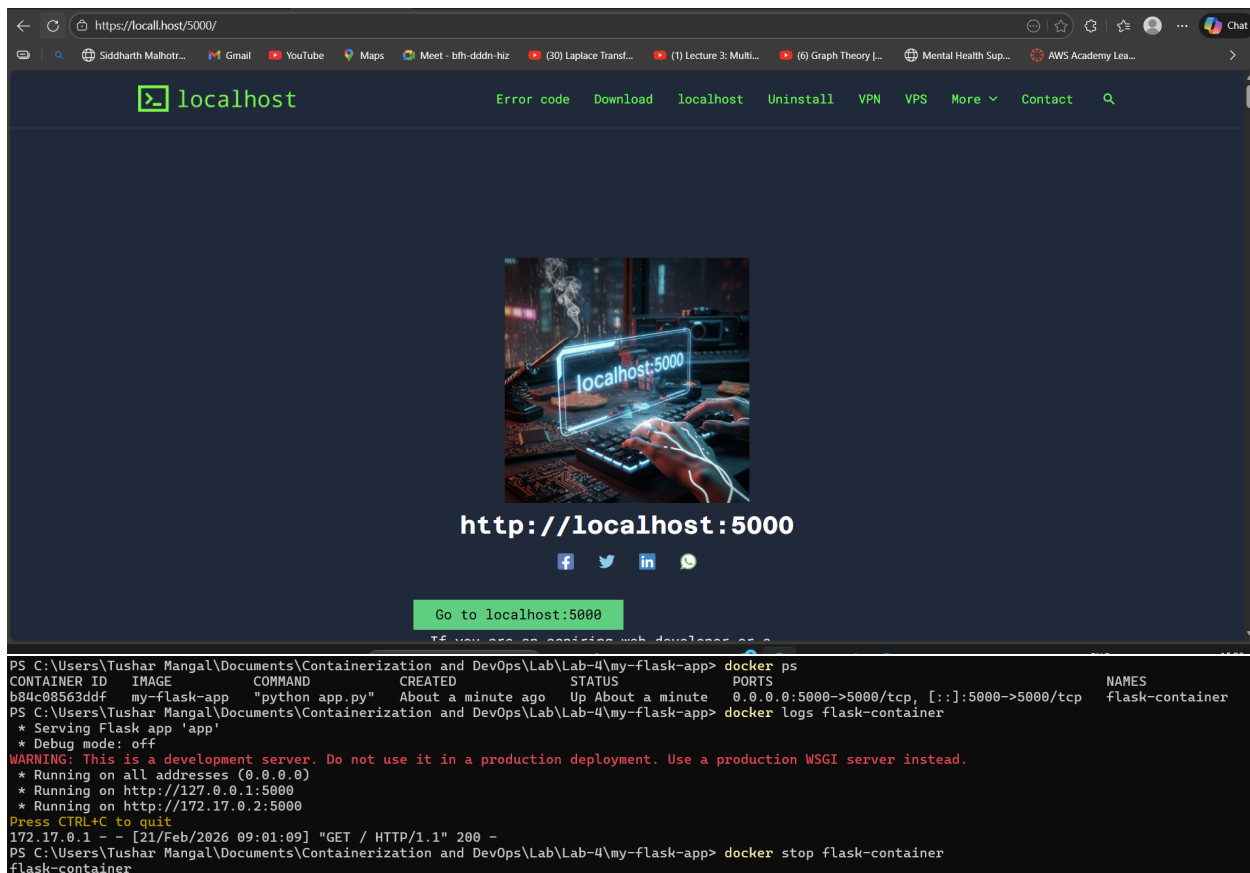
# View container logs
docker logs flask-container

```

```

PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-4\my-flask-app> docker run -d -p 5000:5000 --name flask-container my-flask-app
b84c08563ddf12d17bb08a7bb460be1005f6f864645717bccc9cb32424ade8b

```



10.5.2 Step 2: Manage Containers

Stop container

```
docker stop flask-container
```

Start stopped container

```
docker start flask-container
```

Remove container

```
docker rm flask-container
```

Remove container forcefully

```
docker rm -f flask-container
```

```
PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-4\my-flask-app> docker stop flask-container
flask-container
PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-4\my-flask-app> docker start flask-container
flask-container
PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-4\my-flask-app> docker rm -f flask-container
flask-container
```

10.6 Part 5: Multi-stage Builds

10.6.1 Step 1: Why Multi-stage Builds?

- Smaller final image size
- Better security (remove build tools)
- Separate build and runtime environments

10.6.2 Step 2: Simple Multi-stage Dockerfile

Dockerfile.multistage:

```
# STAGE 1: Builder stage
FROM python:3.9-slim AS builder

WORKDIR /app

# Copy requirements
COPY requirements.txt .

# Install dependencies in virtual environment
RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"
RUN pip install --no-cache-dir -r requirements.txt

# STAGE 2: Runtime stage
FROM python:3.9-slim

WORKDIR /app

# Copy virtual environment from builder
COPY --from=builder /opt/venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"

# Copy application code
COPY app.py .

# Create non-root user
RUN useradd -m -u 1000 appuser
USER appuser

# Expose port
EXPOSE 5000

# Run application
CMD ["python", "app.py"]
```

 Dockerfile.multistage X

 Dockerfile.multistage > ...

```
4  WORKDIR /app
5
6  # Copy requirements
7  COPY requirements.txt .
8
9  # Install dependencies in virtual environment
10 RUN python -m venv /opt/venv
11 ENV PATH="/opt/venv/bin:$PATH"
12 RUN pip install --no-cache-dir -r requirements.txt
13
14 # STAGE 2: Runtime stage
15 FROM python:3.9-slim
16
17 WORKDIR /app
18
19 # Copy virtual environment from builder
20 COPY --from=builder /opt/venv /opt/venv
21 ENV PATH="/opt/venv/bin:$PATH"
22
23 # Copy application code
24 COPY app.py .
25
26 # Create non-root user
27 RUN useradd -m -u 1000 appuser
28 USER appuser
29
30 # Expose port
31 EXPOSE 5000
32
33 # Run application
34 CMD ["python", "app.py"]
35
```

10.6.3 Step 3: Build and Compare

```
# Build regular image
docker build -t flask-regular .

# Build multi-stage image
docker build -f Dockerfile.multistage -t flask-multistage .

# Compare sizes
docker images | findstr flask
```

```
PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-4\my-flask-app> docker build -t flask-regular .
[+] Building 2.6s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> transferring dockerfile: 381B
=> [internal] load metadata for docker.io/library/python:3.9-slim
=> [internal] load .dockerignore
=> transferring context: 304B
=> [1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aablacdale13cf1731b1b
=> resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aablacdale13cf1731b1b
=> [internal] load build context
=> transferring context: 63B
=> CACHED [2/5] WORKDIR /app
=> CACHED [3/5] COPY requirements.txt .
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt
=> CACHED [5/5] COPY app.py .
=> exporting to image
=> exporting layers
=> exporting manifest sha256:c8d2616072f3cdd22c334d17834f3d624565fc106dd2515036cda294b8739873
=> exporting config sha256:47bcef556a96a512e2e965ef7e584562a32fe168e094fe5ebbfce2e07da8904a
=> exporting attestation manifest sha256:4cdcfc15979099a442cb0b7bad07431d034812899c7d35b501f5538ba79736b3f
=> exporting manifest list sha256:b872d11baae281f2eac92ab62e961f50b542b29182d215f6f0c6beddce84928f
=> naming to docker.io/library/flask-regular:latest
=> unpacking to docker.io/library/flask-regular:latest
View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/lm3lgckbtl4ynxw9l6qweenc3
PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-4\my-flask-app> docker build -f Dockerfile.multistage -t flask-multistage .
[+] Building 31.3s (13/13) FINISHED
=> [internal] load build definition from Dockerfile.multistage
=> transferring dockerfile: 709B
=> [internal] load metadata for docker.io/library/python:3.9-slim
=> [internal] load .dockerignore
=> transferring context: 304B
=> [builder 1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aablacdale13cf1731b1b
=> resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aablacdale13cf1731b1b
=> [internal] load build context
=> transferring context: 63B
=> CACHED [builder 2/5] WORKDIR /app
=> CACHED [builder 3/5] COPY requirements.txt .
=> [builder 4/5] RUN python -m venv /opt/venv
=> [builder 5/5] RUN pip install --no-cache-dir -r requirements.txt
=> [stage-1 3/5] COPY --from=builder /opt/venv /opt/venv
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
flask-multistage:latest      633a0182f68d      219MB      52.1MB
flask-regular:latest        b872d11baae2      200MB      48.8MB
my-flask-app:1.0            67ab170e1c73      200MB      48.8MB
my-flask-app:latest         67ab170e1c73      200MB      48.8MB
my-flask-app:v1.0           67ab170e1c73      200MB      48.8MB
username/my-flask-app:1.0   678a8667bb82      200MB      48.8MB
```

10.7 Part 6: Publishing to Docker Hub

10.7.1 Step 1: Prepare for Publishing

```
# Login to Docker Hub
docker login

# Tag image for Docker Hub
docker tag my-flask-app:latest username/my-flask-app:1.0
docker tag my-flask-app:latest username/my-flask-app:latest

# Push to Docker Hub
docker push username/my-flask-app:1.0
docker push username/my-flask-app:latest
```



```
PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-4\my-flask-app> docker login
Authenticating with existing credentials... [Username: usernal]

Info → To login with a different account, run 'docker logout' followed by 'docker login'

Login Succeeded
PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-4\my-flask-app> docker tag my-flask-app:latest usernal/my-flask-app:1.0
PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-4\my-flask-app> docker tag my-flask-app usernal/my-flask-app:latest
PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-4\my-flask-app> docker push usernal/my-flask-app:1.0
The push refers to repository [docker.io/usernal/my-flask-app]
abc5391baad: Already exists
fc7443084902: Layer already exists
b94d5f0ea6f3: Layer already exists
16b6cba43be5: Layer already exists
979b801f22d6: Layer already exists
ff6d87c41c49: Layer already exists
38513bd72563: Layer already exists
ea56f685404a: Layer already exists
b3ec39b36ae8: Layer already exists
1.0: digest: sha256:67ab170e1c73b28dd6107a293781d17f87e045df9bf0683f1a5b4b74efd7d487 size: 856
PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-4\my-flask-app> docker push usernal/my-flask-app:latest
The push refers to repository [docker.io/usernal/my-flask-app]
ff6d87c41c49: Pushed
fc7443084902: Pushed
38513bd72563: Pushed
b94d5f0ea6f3: Pushed
16b6cba43be5: Pushed
abc5391baad: Pushed
b3ec39b36ae8: Pushed
ea56f685404a: Pushed
079b801f22d6: Pushed
latest: digest: sha256:67ab170e1c73b28dd6107a293781d17f87e045df9bf0683f1a5b4b74efd7d487 size: 856
```

10.7.2 Step 2: Pull and Run from Docker Hub

Pull from Docker Hub (on another machine)

```
docker pull username/my-flask-app:latest
```

Run the pulled image

```
docker run -d -p 5000:5000 username/my-flask-app:latest
```

```
PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-4\my-flask-app> docker pull usernal/my-flask-app:latest
latest: Pulling from usernal/my-flask-app
Digest: sha256:67ab170e1c73b28dd6107a293781d17f87e045df9bf0683f1a5b4b74efd7d487
Status: Image is up to date for usernal/my-flask-app:latest
docker.io/usernal/my-flask-app:latest
PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-4\my-flask-app> docker run -d -p 5000:5000 usernal/my-flask-app:latest
53270aa75836bd0b71b87a62e807419d730b4c6bcb6bd4f5e7f3b4357d449655
```

10.8 Part 7: Node.js Example (Quick Version)

10.8.1 Step 1: Node.js Application

```
mkdir my-node-app
```

```
cd my-node-app
```

```
PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-4> mkdir my-node-app

Directory: C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-4

Mode                LastWriteTime         Length Name
----                -
d-----          21-02-2026   15:04                my-node-app

PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-4> cd my-node-app
```

```
app.js:
```

```
const express = require('express');
```

```
const app = express();
```

```
const port = 3000;
```

```
app.get('/', (req, res) => {
```

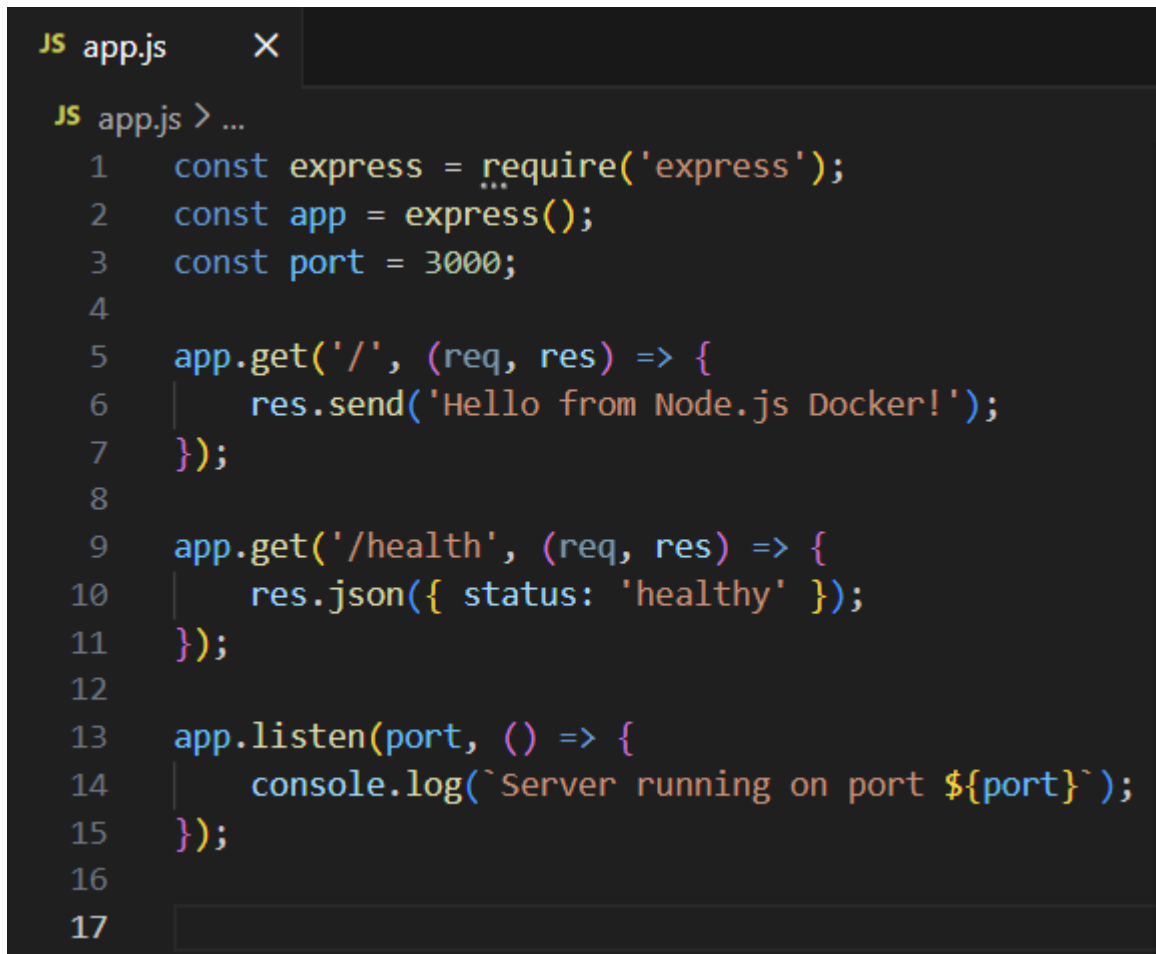
```

    res.send('Hello from Node.js Docker!');
  });

app.get('/health', (req, res) => {
  res.json({ status: 'healthy' });
});

app.listen(port, () => {
  console.log(`Server running on port ${port}`);
});

```



```

JS app.js X
JS app.js > ...
1  const express = require('express');
2  const app = express();
3  const port = 3000;
4
5  app.get('/', (req, res) => {
6    res.send('Hello from Node.js Docker!');
7  });
8
9  app.get('/health', (req, res) => {
10   res.json({ status: 'healthy' });
11 });
12
13 app.listen(port, () => {
14   console.log(`Server running on port ${port}`);
15 });
16
17

```

package.json:

```

{
  "name": "node-docker-app",
  "version": "1.0.0",
  "main": "app.js",
  "dependencies": {
    "express": "^4.18.2"
  }
}

```

```
{ } package.json X
{ } package.json > ...
1  {
2    "name": "node-docker-app",
3    "version": "1.0.0",
4    "main": "app.js",
5    "dependencies": {
6      "express": "^4.18.2"
7    }
8  }
9
```

10.8.2 Step 2: Node.js Dockerfile

```
FROM node:18-alpine
```

```
WORKDIR /app
```

```
COPY package*.json ./
```

```
RUN npm install --only=production
```

```
COPY app.js .
```

```
EXPOSE 3000
```

```
CMD ["node", "app.js"]
```

```
Dockerfile X
Dockerfile > ...
1 FROM node:18-alpine
2
3 WORKDIR /app
4
5 COPY package*.json ./
6 RUN npm install --only=production
7
8 COPY app.js .
9
10 EXPOSE 3000
11
12 CMD ["node", "app.js"]
13
```

10.8.3 Step 3: Build and Run

Build image

```
docker build -t my-node-app .
```

Run container

```
docker run -d -p 3000:3000 --name node-container my-node-app
```

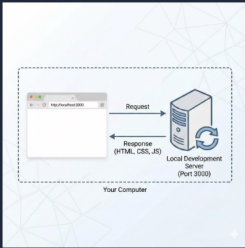
Test

```
curl http://localhost:3000
```

```
PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab-4\my-node-app> docker build -t my-node-app .
[+] Building 34.0s (11/11) FINISHED
=> [internal] load build definition from Dockerfile
=> transferring dockerfile: 194B
=> [internal] load metadata for docker.io/library/node:18-alpine
=> [auth] library/node:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> transferring context: 2B
=> [1/5] FROM docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e
=> resolve docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e
=> sha256:25ff2da83641908f65c3a74d80489d6b1b62ccfaab220b9ea70b80df5a2e0549 446B / 446B
=> sha256:1e5a4c89cee5c0826c540ab86d4b6b491c96eda01837f430bd47f0d26702d6e3 1.26MB / 1.26MB
=> sha256:dd71dde834b5c203d162902e6b8994cb2309ae049a0eabc4feal61b2b5a3d0e 40.01MB / 40.01MB
=> sha256:f18232174bc91741fd3da96d85011092101a032a93a388b79e99e69c2d5c870 3.64MB / 3.64MB
=> extracting sha256:f18232174bc91741fd3da96d85011092101a032a93a388b79e99e69c2d5c870
=> extracting sha256:dd71dde834b5c203d162902e6b8994cb2309ae049a0eabc4feal61b2b5a3d0e
=> extracting sha256:1e5a4c89cee5c0826c540ab86d4b6b491c96eda01837f430bd47f0d26702d6e3
=> extracting sha256:25ff2da83641908f65c3a74d80489d6b1b62ccfaab220b9ea70b80df5a2e0549
=> [internal] load build context
=> transferring context: 541B
=> [2/5] WORKDIR /app
=> [3/5] COPY package*.json ./
=> [4/5] RUN npm install --only=production
=> [5/5] COPY app.js .
=> exporting to image
=> exporting layers
=> exporting manifest sha256:3f6864a4b8501936e7d7646c5d037ff1a3fc0608a71d77702470134be9da6222
=> exporting config sha256:c97554e0439a47e44770a2fdea8f50713bb74c0b5d7be6403938a2b1f464b823
=> exporting attestation manifest sha256:8542c41369bc7de48cd43e6f656b578c5e81b604150f11d17a34db4b0b780414
=> exporting manifest list sha256:ad8f8724c62316abacb2d46079a142caf36c60791ad3015bbee4f406a86045b
=> naming to docker.io/library/my-node-app:latest
=> unpacking to docker.io/library/my-node-app:latest
View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/mez0xmmoyor5a3xuww39511qj
PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab-4\my-node-app> docker run -d -p 3000:3000 --name node-container my-node-app
0ce9efall1ffb0195f26a34a9535ec26db06cc82592d82c71cb43ec63a5b77f5c
```

localhost

Error code Download localhost Uninstall VPN VPS More Contact



http://localhost:3000

Go to localhost:3000

To the world of web development, localhost:3000

11 Containerization-and-DevOps

12 Lab 5: Docker Volumes, Environment Variables, Monitoring & Networks

12.1 Objective

Learn advanced Docker concepts including persistent data storage with volumes, application configuration with environment variables, container monitoring, and linking multiple containers using Docker networks.

12.2 Part 1: Docker Volumes – Persistent Data Storage

By default, data inside a container is ephemeral (lost when the container is removed). Volumes provide a way to safely persist data.

12.2.1 1. Anonymous & Named Volumes

```
docker run -d -v /app/data --name web1 nginx
docker volume create mydata
docker run -d -v mydata:/app/data --name web2 nginx
docker volume ls
```

```
PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-5> docker run -d -v /app/data --name web1 nginx
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
448ea5cac5d5: Pull complete
5435b2dcd5c: Pull complete
7b5d674621c2: Pull complete
84e114c2bb36: Pull complete
054715a6bffa: Pull complete
4a038fd18db1: Pull complete
88d1d984b765: Pull complete
3eff0a97d435: Download complete
cc0cf959117b: Download complete
Digest: sha256:7f0adca1fc6c29c8dc49a2e90037a10ba20dc266baaed0988e9fb4d0d8b85ba0
Status: Downloaded newer image for nginx:latest
c0f2f3083352ae9f59c351f1f4bc844e6ceaed355d383acc36e6c49623dba100
PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-5> docker volume create mydata
mydata
PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-5> docker run -d -v mydata:/app/data --name web2 nginx
db76aaf4574ffadb6accacbe42547494d339608cac2a03bb13705561e0905b57
PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-5> docker volume ls
DRIVER      VOLUME NAME
local       78e3658005c9633c191021bfb9bf73c4706f897e0900622451e3c1e3eea2845a
local       assingment1_postgres_data
local       assingment-1_pgdata
local       mydata
```

12.2.2 2. Bind Mounts (Host Directory)

Bind mounts attach a specific directory on your host machine to a directory inside the container.

```
mkdir myapp-data
docker run -d -v ${PWD}/myapp-data:/app/data --name web3 nginx
echo "From Host" > myapp-data/Text-file.txt
docker exec web3 cat /app/data/Text-file.txt
```

```
PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-5> mkdir myapp-data

Directory: C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-5


Mode                LastWriteTime         Length Name
----                -
d-----           09-04-2026     10:52         myapp-data

PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-5> docker run -d -v ${PWD}/myapp-data:/app/data --name web3 nginx
d325e4eb7b76c7f289a04398f122732ad516a77d592a57909e0092d807b92d94
PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-5> echo "The Text File" > myapp-data/Text-file.txt
PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-5> docker exec web3 cat /app/data/Text-file.txt
♦♦The Text File
```

12.3 Part 2: Environment Variables

Environment variables allow us to pass configuration parameters into the container at runtime.

12.3.1 1. Building and Running a Flask App with ENV Vars

We built a custom Flask application that returns configuration data via a `/config` API endpoint.

`app.py`

```

import os
from flask import Flask
app = Flask(__name__)

db_host = os.environ.get('DATABASE_HOST', 'localhost')
debug_mode = os.environ.get('DEBUG', 'false').lower() == 'true'
api_key = os.environ.get('API_KEY')

@app.route('/config')
def config():
    return {
        'db_host': db_host,
        'debug': debug_mode,
        'has_api_key': bool(api_key)
    }

if __name__ == '__main__':
    port = int(os.environ.get('PORT', 5000))
    app.run(host='0.0.0.0', port=port, debug=debug_mode)

```

Lab > Lab-5 > flask-env >  app.py > ...

```

1  import os
2  from flask import Flask
3  app = Flask(__name__)
4
5  db_host = os.environ.get('DATABASE_HOST', 'localhost')
6  debug_mode = os.environ.get('DEBUG', 'false').lower() == 'true'
7  api_key = os.environ.get('API_KEY')
8
9  @app.route('/config')
10 def config():
11     return {
12         'db_host': db_host,
13         'debug': debug_mode,
14         'has_api_key': bool(api_key)
15     }
16
17 if __name__ == '__main__':
18     port = int(os.environ.get('PORT', 5000))
19     app.run(host='0.0.0.0', port=port, debug=debug_mode)

```

Dockerfile

FROM python:3.9-slim

ENV PYTHONUNBUFFERED=1

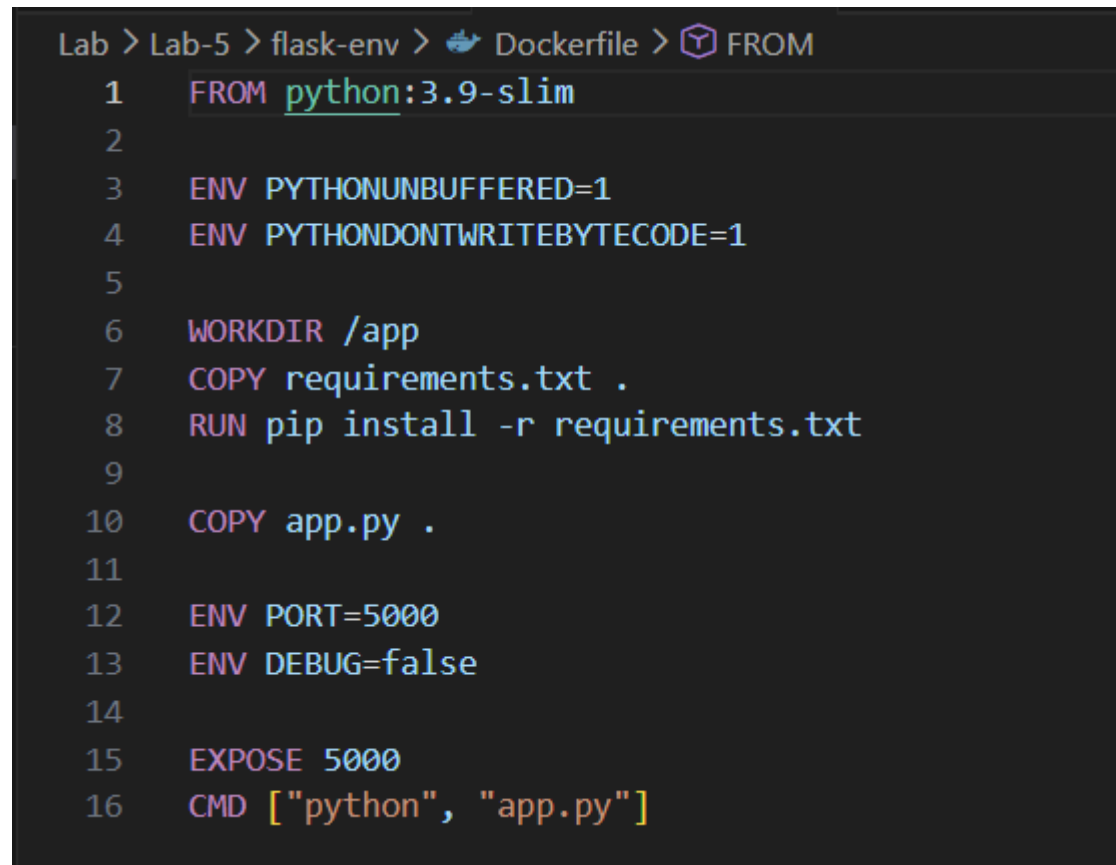
ENV PYTHONDONTWRITEBYTECODE=1

```
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt

COPY app.py .

ENV PORT=5000
ENV DEBUG=false

EXPOSE 5000
CMD ["python", "app.py"]
```

A screenshot of a terminal window with a dark background. The terminal shows the path 'Lab > Lab-5 > flask-env > Dockerfile >' followed by a 'FROM' command. Below this, there is a numbered list of 16 lines of Dockerfile instructions, each preceded by a line number from 1 to 16. The instructions are: 1 FROM python:3.9-slim, 2, 3 ENV PYTHONUNBUFFERED=1, 4 ENV PYTHONDONTWRITEBYTECODE=1, 5, 6 WORKDIR /app, 7 COPY requirements.txt ., 8 RUN pip install -r requirements.txt, 9, 10 COPY app.py ., 11, 12 ENV PORT=5000, 13 ENV DEBUG=false, 14, 15 EXPOSE 5000, 16 CMD ["python", "app.py"].

```
Lab > Lab-5 > flask-env > Dockerfile > FROM
1  FROM python:3.9-slim
2
3  ENV PYTHONUNBUFFERED=1
4  ENV PYTHONDONTWRITEBYTECODE=1
5
6  WORKDIR /app
7  COPY requirements.txt .
8  RUN pip install -r requirements.txt
9
10 COPY app.py .
11
12 ENV PORT=5000
13 ENV DEBUG=false
14
15 EXPOSE 5000
16 CMD ["python", "app.py"]
```

```
docker build -t flask-env-app .
docker run -d --name flask-app -p 5000:5000 \
-e DATABASE_HOST="prod-db.example.com" \
-e DEBUG="true" \
-e API_KEY="secret-xyz" \
flask-env-app
```



```

PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-5\flask-env> docker build -t flask-env .
[+] Building 39.0s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 286B
=> [internal] load metadata for docker.io/library/python:3.9-slim
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aablacdale13c
=> => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aablacdale13c
=> => sha256:fc74430849022d13b0d44b8969a953f842f59c6e9d1a0c2c83d710affa286c08 13.88MB / 13.88MB
=> => sha256:ea56f685404df81680322f152d2cfe62115b30dda481c2c450078315beb508 251B / 251B
=> => sha256:b3ec39b36ae8c03a3e09854de4ec4aa08381dfe84a9daa075048c2e3df3881d 1.29MB / 1.29MB
=> => sha256:38513bd7256313495cdd83b3b0915a633cfa475dc2a07072ab2c8d191020ca5d 29.78MB / 29.78MB
=> => extracting sha256:38513bd7256313495cdd83b3b0915a633cfa475dc2a07072ab2c8d191020ca5d
=> => extracting sha256:b3ec39b36ae8c03a3e09854de4ec4aa08381dfe84a9daa075048c2e3df3881d
=> => extracting sha256:fc74430849022d13b0d44b8969a953f842f59c6e9d1a0c2c83d710affa286c08
=> => extracting sha256:ea56f685404df81680322f152d2cfe62115b30dda481c2c450078315beb508
=> [internal] load build context
=> => transferring context: 85B
=> [2/5] WORKDIR /app
=> [3/5] COPY requirements.txt .
=> [4/5] RUN pip install -r requirements.txt
=> [5/5] COPY app.py .
=> => exporting to image
=> => exporting layers
=> => exporting manifest sha256:d3666df891d65fe6aaa60c715a2aa8df7d8b0e64430bb3daf60a7f78aa17032
=> => exporting config sha256:d012cfcf51alc37372e6aa95d08a2617715e21653b474316df580dffb530356
=> => exporting attestation manifest sha256:6a46395c51f27a3af55830005c442a4e9f1d4ccf9473b29f435c392f3cb6ebd
=> => exporting manifest list sha256:2675e06f84047fcb1af42e562e6f857942636b9f5bb8e0cc7346961c134c354
=> => naming to docker.io/library/flask-env:latest
=> => unpacking to docker.io/library/flask-env:latest

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/SaguSypbn08rh4pwct9bi5p5i
PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-5\flask-env> docker run -d --name flask-app -p 5000:5000 \
docker: invalid reference format

Run 'docker run --help' for more information
PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-5\flask-env> docker run -d --name flask-app -p 5000:5000 -e DATABASE_HOST="prod-db.e
xample.com" -e DEBUG="true" -e API_KEY="secret-xyz" flask-env
0e2d21bb3ddeb2a19fa03b272d05a68f7370ac550f875cdefa514f7fc38a9d22

```

12.3.2 2. Testing Injected Environment Variables

We can verify that the environment variables were successfully passed into the running container.

```

docker exec flask-app printenv DATABASE_HOST
curl http://localhost:5000/config

```

```

PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-5\flask-env> docker exec flask-app printenv DATABASE_HOST
prod-db.example.com
PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-5\flask-env> curl http://localhost:5000/config

Security Warning: Script Execution Risk
Invoke-WebRequest parses the content of the web page. Script code in the web page might be run when the page is parsed.
RECOMMENDED ACTION:
Use the -UseBasicParsing switch to avoid script code execution.

Do you want to continue?

[Y] Yes [A] Yes to All [N] No [L] No to All [S] Suspend [?] Help (default is "N"): y

StatusCode      : 200
StatusDescription : OK
Content          : {
  "db_host": "prod-db.example.com",
  "debug": true,
  "has_api_key": true
}

RawContent       : HTTP/1.1 200 OK
                  Connection: close
                  Content-Length: 79
                  Content-Type: application/json
                  Date: Thu, 09 Apr 2026 05:33:42 GMT
                  Server: Werkzeug/3.1.8 Python/3.9.25

                  {
                    "db_host": "prod-db.example.com..."
                  }

Forms            : {}
Headers          : {[Connection, close], [Content-Length, 79], [Content-Type, application/json], [Date, Thu, 09 Apr 2026 05:33:42 GMT]...}
Images           : {}
InputFields      : {}
Links            : {}
ParsedHtml       : mshtml.HTMLDocumentClass
RawContentLength : 79

```

12.4 Part 3: Docker Monitoring

Monitoring allows us to keep track of resource usage, running processes, and application logs.

```
docker stats --no-stream
docker top flask-app
docker logs flask-app
```

```
PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-5\flask-env> docker stats --no-stream
CONTAINER ID   NAME      CPU %     MEM USAGE / LIMIT   MEM %     NET I/O       BLOCK I/O   PIDS
0e2d21bb3dde   flask-app  0.30%    42.43MiB / 7.45GiB   0.56%     1.57kB / 858B   0B / 0B     3
d325e4eb7b76   web3      0.00%    10.41MiB / 7.45GiB   0.14%     1.21kB / 126B   4.08MB / 12.3kB 13
db76aaf4574f   web2      0.00%    10.46MiB / 7.45GiB   0.14%     1.46kB / 126B   0B / 12.3kB     13
c0f2f3083352   web1      0.00%    10.44MiB / 7.45GiB   0.14%     2.41kB / 126B   0B / 12.3kB     13
81108b4c493d   node_backend 0.00%    18.4MiB / 7.45GiB   0.24%     2.73kB / 1.24kB 578kB / 0B      11
e873da160789   postgres_db 0.00%    30.79MiB / 7.45GiB   0.40%     1.8kB / 1.89kB  42MB / 303kB     6
PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-5\flask-env> docker top flask-app
UID            PID            PPID           C              STIME          TTY            TIME           CMD
root           4321           4299           1              05:32          ?              00:00:01      python app.py
root           4343           4321           1              05:32          ?              00:00:01      /usr/local/bin/p
ython /app/app.py
PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-5\flask-env> docker logs flask-app
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.17.0.5:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 104-558-911
172.17.0.1 - - [09/Apr/2026 05:33:42] "GET /config HTTP/1.1" 200 -
```

12.4.1 Automated Monitoring Script

We can automate these checks using a bash script (`monitor.sh`):

monitor.sh

```
#!/bin/bash
```

```
echo "=== Docker Monitoring Dashboard ==="
```

```
echo "Time: $(date)"
```

```
echo "1. Running Containers:"
```

```
docker ps --format "table \t\t"
```

```
echo "2. Resource Usage:"
```

```
docker stats --no-stream --format "table \t\t"
```

```
echo "3. Recent Events:"
```

```
docker events --since '5m' --until '0s' | tail
```

```
echo "4. System Info:"
```

```
docker system df
```

```

Lab > Lab-5 > $ monitor.sh
1  #!/bin/bash
2
3  echo "=== Docker Monitoring Dashboard ==="
4  echo "Time: $(date)"
5
6  echo "1. Running Containers:"
7  docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
8
9  echo "2. Resource Usage:"
10 docker stats --no-stream --format "table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}"
11
12 echo "3. Recent Events:"
13 docker events --since '5m' --until '0s' | tail
14
15 echo "4. System Info:"
16 docker system df

```

12.5 Part 4: Docker Networks

Docker networks allow isolated containers to securely communicate with each other.

12.5.1 1. Creating a Network

We created a custom bridge network and attached two Nginx containers to it.

```

docker network create my-network
docker run -d --name net-web1 --network my-network nginx
docker run -d --name net-web2 --network my-network nginx

```

```

PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-5\flask-env> docker network create my-network
836ff7025198ea7a06e0df96b0fe8e06d3eaa73bbad44656e7ae43ce2a8c2a07
PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-5\flask-env> docker run -d --name net-web1 --network my-network nginx
620c6532ae622799ffe895275165aa8dcfb79945709f15690d41f01be25ea354
PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-5\flask-env> docker run -d --name net-web2 --network my-network nginx
ae9151683ab7c826983516761d5caa28695f0071191cddd89267e15798f64066

```

12.5.2 2. Internal Network Communication

Because they share a custom network, containers can communicate with each other using their container names as DNS addresses.

```

docker exec net-web1 curl -s http://net-web2

```

```
PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-5\flask-env> docker exec net-web1 curl -s http://net-web2
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, nginx is successfully installed and working.
Further configuration is required for the web server, reverse proxy,
API gateway, load balancer, content cache, or other features.</p>

<p>For online documentation and support please refer to
<a href="https://nginx.org/">nginx.org</a>.<br/>
To engage with the community please visit
<a href="https://community.nginx.org/">community.nginx.org</a>.<br/>
For enterprise grade support, professional services, additional
security features and capabilities please refer to
<a href="https://f5.com/nginx">f5.com/nginx</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```

12.6 Part 5: Real-World Example (Multi-Container Setup)

In a real-world scenario, we often have an application connected to a database and a cache layer, all residing on the same network.

12.6.1 1. Running Postgres and Redis on a Shared Network

```
docker network create myapp-network
docker run -d --name postgres --network myapp-network \
-e POSTGRES_PASSWORD=mysecretpassword \
-v postgres-data:/var/lib/postgresql/data postgres:15
```

```
docker run -d --name redis --network myapp-network \
-v redis-data:/data redis:7-alpine
```

```
PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-5\flask-env> docker network create myapp-network
481903d333ce683b9a1d1ca9fc176673edbaa98c173e68efc8889d72dd0fa68
PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-5\flask-env> docker run -d --name postgres --network myapp-network -e POSTGRES_PASSW
ORD=mysecretpassword -v postgres-data:/var/lib/postgresql/data postgres:15
Unable to find image 'postgres:15' locally
15: Pulling from library/postgres
71a32c7571f4: Pull complete
a8a0725a32a6: Pull complete
5c152148754e: Pull complete
b4765ac2e7e6: Pull complete
af2a6e5f285c: Pull complete
5ca81a59b0a4: Pull complete
00f33d580dc3: Pull complete
8ecd8d190da6: Pull complete
c45b346571fc: Pull complete
2c38b75ffca2: Pull complete
af0c3ea2fe5e: Pull complete
9328c154beae: Pull complete
f7049a93e93f: Pull complete
cebcb0a7837b: Download complete
b9015a4f6b87: Download complete
Digest: sha256:31224c60946ea75ee9522e0f4d02eb202e74754adea4294fdefe9f93c875b657
Status: Downloaded newer image for postgres:15
30b3e79045ae62962194fce9f6df811c8295115038146aad14308615d78c338e2
PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-5\flask-env> docker run -d --name redis --network myapp-network -v redis-data:/data
redis:7-alpine
Unable to find image 'redis:7-alpine' locally
7-alpine: Pulling from library/redis
1c2c8d1ee428: Pull complete
09a5a0c32a23: Pull complete
7ad54b3c4cef: Pull complete
4f4fb700ef54: Pull complete
9fe3744a2eac: Pull complete
2a97533d89ca: Pull complete
bd53938b1271: Pull complete
bc1da058f299: Pull complete
9bbcf5d5580: Download complete
06cd2b88db48: Download complete
Digest: sha256:8b81dd37f7f027bec9e516d41acfb9e9fe2460070dc6d4a4570a2ac5b9d59df065
Status: Downloaded newer image for redis:7-alpine
b86bfff5f8551a000e5fb997aa1caca301215742bd33e172d6fc6c954ad5a2cfb
```

12.6.2 2. Verifying the Services

```
docker ps
```

```
docker stats --no-stream postgres redis
```

```
PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-5\flask-env> docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAME
b86bff5f8551	redis:7-alpine	"docker-entrypoint.s..."	3 seconds ago	Up 2 seconds	6379/tcp	redi
38b3e7945ae6	postgres:15	"docker-entrypoint.s..."	About an hour ago	Up About an hour	5432/tcp	post
ae9151683ab7	nginx	"/docker-entrypoint..."	2 hours ago	Up 2 hours	80/tcp	net-
620c6532ae62	nginx	"/docker-entrypoint..."	2 hours ago	Up 2 hours	80/tcp	net-
0e2d21bb3dde	flask-env	"python app.py"	2 hours ago	Up 2 hours	0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp	flas
d325e4eb7b76	nginx	"/docker-entrypoint..."	2 hours ago	Up 2 hours	80/tcp	web3
db76aaf4574f	nginx	"/docker-entrypoint..."	2 hours ago	Up 2 hours	80/tcp	web2
c0f2f3083352	nginx	"/docker-entrypoint..."	2 hours ago	Up 2 hours	80/tcp	web1
81108b4c493d	assignment-1-backend	"docker-entrypoint.s..."	2 weeks ago	Up 2 hours	0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp	node
0e873da160789	assignment-1-db	"docker-entrypoint.s..."	2 weeks ago	Up 2 hours (healthy)		post

```
gres_db
PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-5\flask-env> docker stats --no-stream postgres redis
```

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
38b3e7945ae6	postgres	0.05%	35.21MiB / 7.45GiB	0.46%	1.82kB / 126B	9.69MB / 40.8MB	6
b86bff5f8551	redis	0.76%	3.539MiB / 7.45GiB	0.05%	872B / 126B	0B / 0B	6

12.7 Cleanup

```
docker stop $(docker ps -aq)
```

```
docker rm $(docker ps -aq)
```

```
docker volume prune -f
```

```
docker network prune -f
```

```
PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-5\flask-env> docker stop $(docker ps -aq)
b86bff5f8551
38b3e7945ae6
ae9151683ab7
620c6532ae62
0e2d21bb3dde
d325e4eb7b76
db76aaf4574f
c0f2f3083352
81108b4c493d
e873da160789
PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-5\flask-env> docker rm $(docker ps -aq)
b86bff5f8551
38b3e7945ae6
ae9151683ab7
620c6532ae62
0e2d21bb3dde
d325e4eb7b76
db76aaf4574f
c0f2f3083352
81108b4c493d
e873da160789
PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-5\flask-env> docker volume prune -f
Deleted Volumes:
78e3658005c9633c191021bf9bf73c4706f897e0900622451e3c1e3eea2845a

Total reclaimed space: 0B
PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-5\flask-env> docker network prune -f
Deleted Networks:
myapp-network
assignment-1_bridge_net
my_ipvlan_net
my-network
```

12.8 Docker Commands Cheat Sheet

12.8.1 Volumes

Command	Description	——— ———	docker volume create <name>	Create a named volume
docker run -v <volume>:/path	Mount named volume	docker run -v /host:/container	Mount local bind	

12.8.2 Environment Variables

Command	Description
<code>docker run -e VAR=value</code>	Inject single variable
<code>docker run --env-file .env</code>	Inject multiple from file

12.8.3 Monitoring

Command	Description
<code>docker stats</code>	Live resource usage
<code>docker top <container></code>	process list inside container
<code>docker logs -f <container></code>	Follow container logs

12.8.4 Networks

Command	Description
<code>docker network create <name></code>	Create custom network
<code>docker run --network <name></code>	Run container on network

12.9 Key Takeaways

1. **Volumes persist data** safely outside the container lifecycle.
2. **Env variables configure apps** without modifying the actual code or image.
3. **Monitoring helps debugging** performance, memory issues, and hidden errors.
4. **Networks enable communication** securely via DNS isolation between containers.
5. Use **named volumes in production** for database persistence.
6. Use **custom networks** for multi-container applications instead of linking containers manually.

13 Containerization-and-DevOps

14 Experiment 6A: Comparison of Docker Run and Docker Compose

14.1 PART A – THEORY

14.1.1 1. Objective

To understand the relationship between Docker Run and Docker Compose, and compare their syntax and use cases.

14.1.2 2. Background Theory

14.1.2.1 2.1 Docker Run (Imperative Approach)

- Used to create and start containers.
- Requires manual flags for everything (Ports, Volumes, Networks, Restart policies, Resources, Names).

Example:

```
docker run -d \
  --name my-nginx \
  -p 8080:80 \
  -v ./html:/usr/share/nginx/html \
  -e NGINX_HOST=localhost \
  --restart unless-stopped \
  nginx:alpine
```

14.1.2.2 2.2 Docker Compose (Declarative Approach)

- Uses `docker-compose.yml` to define services, networks, and volumes in a single file.
- One command (`docker compose up -d`) runs everything.

Equivalent Compose File:

```
version: '3.8'
services:
  nginx:
    image: nginx:alpine
    container_name: my-nginx
    ports:
      - "8080:80"
    volumes:
      - ./html:/usr/share/nginx/html
    environment:
      NGINX_HOST: localhost
    restart: unless-stopped
```

14.1.3 3. Mapping: Docker Run vs Compose

Docker Run	Docker Compose
<code>-p</code>	<code>ports</code>
<code>-v</code>	<code>volumes</code>
<code>-e</code>	<code>environment</code>
<code>--name</code>	<code>container_name</code>
<code>--network</code>	<code>networks</code>
<code>--restart</code>	<code>restart</code>
<code>--memory</code>	<code>deploy.resources.limits.memory</code>
<code>--cpus</code>	<code>deploy.resources.limits.cpus</code>

14.1.4 4. Advantages of Docker Compose

- Multi-container support
- Reproducibility & Version control
- Unified lifecycle commands
- Scaling capability (`docker compose up --scale web=3`)

14.2 PART B – PRACTICAL TASKS

14.2.1 Task 1: Single Container Comparison

14.2.1.1 Using Docker Run

```
docker run -d --name lab-nginx -p 8081:80 -v $(pwd)/html:/usr/share/nginx/html nginx:alpine
curl http://localhost:8081
```

```

PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-6\task1-single> echo "Hello docker"> html/index.html
PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-6\task1-single> docker run -d --name lab-nginx -p 8081:80 -v ${PWD}/html:/usr/share/
nginx/html nginx:alpine
Unable to find image 'nginx:alpine' locally
alpine: Pulling from library/nginx
ff9f59a6a62e: Pull complete
1165b869c51a: Pull complete
a71873b303e8: Pull complete
15e759724ff6: Pull complete
34dfdd2ef1f9: Pull complete
f03becc8ac15: Pull complete
c8a2fa3a88d2: Pull complete
6057082906d9: Download complete
1116431dccff: Download complete
Digest: sha256:645eda1c2477aa9b879f73909b9222c6f19798dd45be6706268d82a661c6e6d
Status: Downloaded newer image for nginx:alpine
6218ed65efd04d9332729c3b421ed18f0f2846d50dd46a923991ab5cabe49667
PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-6\task1-single> curl http://localhost:8081

Security Warning: Script Execution Risk
Invoke-WebRequest parses the content of the web page. Script code in the web page might be run when the page is parsed.
RECOMMENDED ACTION:
Use the -UseBasicParsing switch to avoid script code execution.

Do you want to continue?

[Y] Yes [A] Yes to All [N] No [L] No to All [S] Suspend [?] Help (default is "N"): y

StatusCode      : 200
StatusDescription : OK
Content          : ybHello docker

RawContent       : HTTP/1.1 200 OK
                   Connection: keep-alive
                   Accept-Ranges: bytes
                   Content-Length: 30
                   Content-Type: text/html
                   Date: Thu, 09 Apr 2026 08:57:37 GMT
                   ETag: "69d769e5-1e"
                   Last-Modified: Thu, 09 Apr 2026 08...
                   Last-Modified: Thu, 09 Apr 2026 08...
Forms            : {}
Headers          : {[Connection, keep-alive], [Accept-Ranges, bytes], [Content-Length, 30], [Content-Type, text/html]...}
Images           : {}
InputFields      : {}
Links            : {}
ParsedHtml       : mshtml.HTMLDocumentClass
RawContentLength : 30

```

14.2.1.2 Using Docker Compose docker-compose.yml

version: '3.8'

services:

nginx:

image: nginx:alpine

container_name: lab-nginx

ports:

- "8081:80"

volumes:

- ./html:/usr/share/nginx/html

docker compose up -d

docker compose ps


```

PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-6\task1-single> docker compose up -d
time="2026-04-09T14:49:02+05:30" level=warning msg="C:\\Users\\Tushar Mangal\\Documents\\Containerization and DevOps\\Lab\\Lab-6\\task1-single\\docker-compo
se.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] up 2/2
  ✓ Network task1-single-default Created 0.1s
  ✓ Container lab-nginx Created 0.2s
PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-6\task1-single> curl http://localhost:8881

Security Warning: Script Execution Risk
Invoke-WebRequest parses the content of the web page. Script code in the web page might be run when the page is parsed.
RECOMMENDED ACTION:
Use the -UseBasicParsing switch to avoid script code execution.

Do you want to continue?

[Y] Yes [A] Yes to All [N] No [L] No to All [S] Suspend [?] Help (default is "N"): y

StatusCode      : 200
StatusDescription : OK
Content          : ybHello docker

RawContent      : HTTP/1.1 200 OK
                  Connection: keep-alive
                  Accept-Ranges: bytes
                  Content-Length: 30
                  Content-Type: text/html
                  Date: Thu, 09 Apr 2026 09:19:25 GMT
                  ETag: "69d769a5-1e"
                  Last-Modified: Thu, 09 Apr 2026 08...

Forms           : {}
Headers         : {[Connection, keep-alive], [Accept-Ranges, bytes], [Content-Length, 30], [Content-Type, text/html]...}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 30

PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-6\task1-single> docker compose ps
time="2026-04-09T14:49:05+05:30" level=warning msg="C:\\Users\\Tushar Mangal\\Documents\\Containerization and DevOps\\Lab\\Lab-6\\task1-single\\docker-compo
se.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
NAME          IMAGE          COMMAND          SERVICE    CREATED      STATUS      PORTS
lab-nginx     nginx:alpine    "/docker-entrypoint..." nginx       42 seconds ago Up 41 seconds 0.0.0.0:8081->80/tcp, [::]:8081->80/tcp

```

14.2.2 Task 2: Multi-Container App (WordPress + MySQL)

14.2.2.1 Using Docker Run (Manual approach)

docker network create wp-net

docker run -d --name mysql --network wp-net -e MYSQL_ROOT_PASSWORD=secret -e MYSQL_DATABASE=wordpress mysql

docker run -d --name wordpress --network wp-net -p 8082:80 -e WORDPRESS_DB_HOST=mysql -e WORDPRESS_DB_P

```

PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-6\task1-single> docker network create wp-net
ea6c8498d440ec224dcad6f9ad7fbf1303e24b5b34493fab599926a40f385a4c
PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-6\task1-single> docker run -d --name mysql --network wp-net -e MYSQL_ROOT_PASSWORD=s
ecret -e MYSQL_DATABASE=wordpress mysql:5.7
Unable to find image 'mysql:5.7' locally
5.7: Pulling from library/mysql
d49a4d85569b: Pull complete
68c3898c2815: Pull complete
20e4dcae4c69: Pull complete
ffc89e9d4d88: Pull complete
43d05e938198: Pull complete
6b95a940e7b6: Pull complete
ae71319cb779: Pull complete
90986bb8de6e: Pull complete
e9f03a1c24ce: Pull complete
1c56c3d4ce74: Pull complete
064b2d298fba: Pull complete
96889c47a44a: Download complete
9eb6a52b8441: Download complete
Digest: sha256:4bc6bc963e6d8443453676cae56536f4b8156d78bae03c0145cbe47c2aad73bb
Status: Downloaded newer image for mysql:5.7
1cb945e39eeb42f7ed8b2baa38c7ffa1614063e168d58cff14a0440d3234259f
PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-6\task1-single> docker run -d --name wordpress --network wp-net -p 8082:80 -e WORDPR
ESS_DB_HOST=mysql -e WORDPRESS_DB_PASSWORD=secret wordpress:latest
Unable to find image 'wordpress:latest' locally
latest: Pulling from library/wordpress
e24121de25e6: Pull complete
90be0470639d: Pull complete
871c6a1a47cc: Pull complete
6562f7228ec3: Pull complete
1fa182c1d458: Pull complete
568912d7e28d: Pull complete
78a70974d738: Pull complete
4443d9c71a47: Pull complete
b4b1305df06a: Pull complete
d5af5d4a3d4d: Pull complete
e052461f1b41: Pull complete
73ce216c5445: Pull complete
79e11af4b256: Pull complete
9bde0c34d1f4: Pull complete
271a7052ce75: Pull complete
PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-6\task1-single> docker compose ps
time="2026-04-09T14:53:03+05:30" level=warning msg="C:\\Users\\Tushar Mangal\\Documents\\Containerization and DevOps\\Lab\\Lab-6\\task1-single\\docker-compo
se.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
NAME          IMAGE          COMMAND          SERVICE    CREATED      STATUS      PORTS
lab-nginx     nginx:alpine    "/docker-entrypoint..." nginx       4 minutes ago Up 4 minutes 0.0.0.0:8081->80/tcp, [::]:8081->80/tcp

```

(Note: The equivalent steps in Compose are demonstrated in Experiment 6B below).

14.3 PART C – CONVERSION TASKS (Theory)

Converting manual `docker run` commands and configurations into declarative `docker-compose.yml` configurations:

14.3.1 Task 3: Convert Run → Compose

Given Run Command:

```
docker run -d --name webapp -p 5000:5000 -e APP_ENV=production -e DEBUG=false --restart unless-stopped
```

Converted Compose File:

```
version: '3.8'
services:
  webapp:
    image: node:18-alpine
    container_name: webapp
    ports:
      - "5000:5000"
    environment:
      APP_ENV: production
      DEBUG: false
    restart: unless-stopped
```

14.3.2 Task 4 & 5: Volumes, Networks, and Resource Limits

Compose clearly organizes dependencies, limits, and custom named networks natively without extra CLI scaffolding. (See lab theory notes for full limit syntax using `deploy:`).

14.4 PART D – DOCKERFILE WITH COMPOSE

We can combine custom image building with Compose orchestration.

app.js

```
const http = require('http');
http.createServer((req, res) => {
  res.end("Docker Compose Build Lab");
}).listen(3000);
```

Dockerfile

```
FROM node:18-alpine
WORKDIR /app
COPY app.js .
EXPOSE 3000
CMD ["node", "app.js"]
```

docker-compose.yml

```
version: '3.8'
services:
  nodeapp:
```

```

build:
  context: .
  dockerfile: Dockerfile
container_name: custom-node-app
ports:
  - "3000:3000"

```

```

docker compose up --build -d
curl http://localhost:3000

```

```

PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-6\part-d-node> docker compose up --build -d
time="2026-04-09T19:44:15+05:30" level=warning msg="C:\\Users\\Tushar Mangal\\Documents\\Containerization and DevOps\\Lab\\Lab-6\\part-d-node\\docker-compos
e.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
#1 [internal] load local bake definitions
#1 reading from stdin 643B done
#1 DONE 0.0s

#2 [internal] load build definition from dockerfile
#2 transferring dockerfile: 122B 0.0s done
#2 DONE 0.1s

#3 [internal] load metadata for docker.io/library/node:18-alpine
#3 DONE 0.1s

#4 [internal] load .dockerignore
#4 transferring context:
#4 transferring context: 2B done
#4 DONE 0.1s

#5 [internal] load build context
#5 transferring context: 155B 0.0s done
#5 DONE 0.1s

#6 [1/3] FROM docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e
#6 resolve docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e 0.1s done
#6 DONE 0.1s

#7 [2/3] WORKDIR /app
#7 CACHED

#8 [3/3] COPY app.js .
#8 DONE 0.1s

#9 exporting to image
#9 exporting layers
#9 exporting layers 0.2s done
#9 exporting manifest sha256:17bc1b78d4d95de13086f27f4092f59fe189716cfaa72d8c608bb8b7d61ef4d2 0.0s done
#9 exporting config sha256:acdb06cedbba30898e2f3b8bd3147f2c6a4c4715a8cbb0d2fa0bc6d007a57484 0.0s done
#9 exporting attestation manifest sha256:9faeb86f7a039aa3609c76db5a8c3b88f546738e0b11130690001147afa9ae6b
#9 exporting attestation manifest sha256:9faeb86f7a039aa3609c76db5a8c3b88f546738e0b11130690001147afa9ae6b 0.1s done
#9 exporting manifest list sha256:197c4bb2e0fda31ae76e00fff9e4603e2cd91d6a29366fc637d67cf7af03452a 0.0s done

#10 resolving provenance for metadata file
#10 DONE 0.0s
[+] up 3/3
✔Image part-d-node-nodeapp Built 2.5s
✔Network part-d-node_default Created 0.1s
✔Container custom-node-app Created 0.2s
PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-6\part-d-node> curl http://localhost:3000

Security Warning: Script Execution Risk
Invoke-WebRequest parses the content of the web page. Script code in the web page might be run when the page is parsed.
RECOMMENDED ACTION:
Use the -UseBasicParsing switch to avoid script code execution.

Do you want to continue?

[Y] Yes [A] Yes to All [N] No [L] No to All [S] Suspend [?] Help (default is "N"): y

StatusCode      : 200
StatusDescription : OK
Content          : {68, 111, 99, 107...}
RawContent       : HTTP/1.1 200 OK
                  Connection: keep-alive
                  Keep-Alive: timeout=5
                  Content-Length: 24
                  Date: Thu, 09 Apr 2026 14:14:56 GMT

                  Docker Compose Build Lab
Headers          : {[Connection, keep-alive], [Keep-Alive, timeout=5], [Content-Length, 24], [Date, Thu, 09 Apr 2026 14:14:56 GMT]}
RawContentLength : 24

PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-6\part-d-node> docker compose ps
time="2026-04-09T19:45:11+05:30" level=warning msg="C:\\Users\\Tushar Mangal\\Documents\\Containerization and DevOps\\Lab\\Lab-6\\part-d-node\\docker-compos
e.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS        PORTS
custom-node-app     part-d-node-nodeapp "docker-entrypoint.s..." nodeapp     53 seconds ago Up 52 seconds 0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp

```

15 Experiment 6B: WordPress + MySQL using Compose

Using a single Compose file completely replaces the manual networking and container execution from Task 2.

docker-compose.yml

```
version: '3.9'

services:
  db:
    image: mysql:5.7
    container_name: wordpress_db
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: rootpass
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wpuser
      MYSQL_PASSWORD: wppass
    volumes:
      - db_data:/var/lib/mysql

  wordpress:
    image: wordpress:latest
    container_name: wordpress_app
    depends_on:
      - db
    ports:
      - "8080:80"
    restart: always
    environment:
      WORDPRESS_DB_HOST: db:3306
      WORDPRESS_DB_USER: wpuser
      WORDPRESS_DB_PASSWORD: wppass
      WORDPRESS_DB_NAME: wordpress
    volumes:
      - wp_data:/var/www/html

volumes:
  db_data:
  wp_data:
```

15.0.1 1. Run the Stack

docker compose up -d

docker ps

```
PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-6\exp6b-wordpress> docker compose up -d
time="2026-04-09T19:47:11+05:30" level=warning msg="C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-6\exp6b-wordpress\docker-co
mpose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[*] up 5/5
✔Network exp6b-wordpress_default Created 0.1s
✔Volume exp6b-wordpress_db_data Created 0.0s
✔Volume exp6b-wordpress_wp_data Created 0.0s
✔Container wordpress_db Created 0.3s
✔Container wordpress_app Created 0.3s
PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-6\exp6b-wordpress> docker compose ps
time="2026-04-09T19:47:24+05:30" level=warning msg="C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-6\exp6b-wordpress\docker-co
mpose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS        PORTS
wordpress_app       wordpress:latest     "docker-entrypoint.s..." wordpress  12 seconds ago Up 11 seconds 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
wordpress_db        mysql:5.7            "docker-entrypoint.s..." db         12 seconds ago Up 11 seconds 3306/tcp, 33060/tcp
```

15.0.2 2. Scaling

docker compose up --scale wordpress=3

```
PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-6\exp6b-wordpress> docker compose up --scale wordpress=3
time="2026-04-09T19:47:40+05:30" level=warning msg="C:\\Users\\Tushar Mangal\\Documents\\Containerization and DevOps\\Lab\\Lab-6\\exp6b-wordpress\\docker-co
mpose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] up 1/1
✓Container wordpress_db Running 0.0s
WARNING: The "wordpress" service is using the custom container name "wordpress_app". Docker requires each container to have a unique name. Remove the custom
name to scale the service
```

Important Learning Outcome: The scaling intentionally throws an error because standard Docker Compose is limited here. Since `container_name: wordpress_app` and `ports: "8080:80"` are hardcoded in the file, Compose cannot launch 3 replicas that all want the identical name and host port. This demonstrates the exact scenario where **Docker Swarm** (which handles dynamic naming and load balances identical ports) becomes necessary for production!

15.1 Docker Compose vs Swarm Comparison

Feature	Compose	Swarm
Scope	Single host	Multi-node
Scaling	Manual / Limited by port bindings	Built-in
Load balancing	No	Yes
Self-healing	No	Yes

15.2 Key Takeaways

1. **Compose simplifies multi-container apps** by keeping settings in version control.
2. **Declarative approach** is much easier than manually tracking `docker run` flags.
3. **Compose is excellent for dev/test**, but limited in native scaling due to port/name collisions without strict dynamic tuning.
4. **Docker Swarm** is much better for production cluster orchestration, load balancing, and self-healing.

16 Containerization-and-DevOps

17 Lab 7: CI/CD using Jenkins, GitHub & Docker Hub

17.1 1. Aim

To build a complete CI/CD pipeline using:

- **GitHub** — Source code repository
- **Jenkins** — Automation server
- **Docker Hub** — Container image storage

17.2 2. Objective

- Understand the CI/CD workflow end-to-end
- Automate the build and push process using Jenkins
- Write a Jenkins pipeline using **Jenkinsfile**
- Store credentials securely inside Jenkins
- Trigger automated builds using GitHub Webhooks

17.3 3. What is CI/CD?

17.3.1 CI – Continuous Integration

Code is automatically built and tested after every commit pushed to GitHub.

17.3.2 CD – Continuous Deployment

The tested application is automatically packaged and deployed without manual steps.

17.3.3 Flow Diagram

Developer → GitHub → Webhook → Jenkins → Docker Hub

17.4 4. What is Jenkins?

Jenkins is a **GUI-based open-source automation server** that enables:

- **Building** applications
- **Testing** code automatically
- **Deploying** to target environments

17.4.1 Key Features

Feature	Description
Web Dashboard	Visual interface to manage jobs and pipelines
Plugin Support	1800+ plugins for every major tool
Pipeline as Code	Define your full CI/CD flow in a Jenkinsfile
Credential Store	Securely handles secrets and tokens

17.5 5. Project Structure

```
jenkins_lab-main/  
  app.py  
  requirements.txt  
  Dockerfile  
  Jenkinsfile
```

17.6 6. Application Code (Flask)

app.py

```
from flask import Flask  
app = Flask(__name__)  
  
@app.route("/")  
def home():  
    return "Hello from CI/CD Pipeline!"  
  
app.run(host="0.0.0.0", port=80)
```

requirements.txt

flask

17.7 7. Dockerfile

```
FROM python:3.10-slim
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
EXPOSE 80
CMD ["python", "app.py"]
```

17.8 8. Jenkinsfile (Pipeline)

The Jenkinsfile defines the full CI/CD pipeline as code:

```
pipeline {
    agent any

    environment {
        IMAGE_NAME = "your-dockerhub-username/myapp"
    }

    stages {
        stage('Clone Source') {
            steps {
                git 'https://github.com/your-username/my-app.git'
            }
        }

        stage('Build Docker Image') {
            steps {
                sh 'docker build -t $IMAGE_NAME:latest .'
            }
        }

        stage('Login to Docker Hub') {
            steps {
                withCredentials([string(credentialsId: 'dockerhub-token', variable: 'DOCKER_TOKEN')]) {
                    sh 'echo $DOCKER_TOKEN | docker login -u your-dockerhub-username --password-stdin'
                }
            }
        }

        stage('Push to Docker Hub') {
            steps {
                sh 'docker push $IMAGE_NAME:latest'
            }
        }
    }
}
```

17.8.1 Pipeline Stages Explained

Stage	What It Does
Clone Source	Pulls the latest code from GitHub
Build Docker Image	Runs <code>docker build</code> to create the image
Login to Docker Hub	Authenticates using stored token credential
Push to Docker Hub	Uploads the built image to Docker Hub

17.9 9. Jenkins Setup using Docker

We run Jenkins itself as a Docker container for easy setup.

`docker-compose.yml`

```
version: "3.8"
services:
  jenkins:
    image: jenkins/jenkins:lts
    container_name: jenkins
    ports:
      - "8080:8080"
      - "50000:50000"
    volumes:
      - jenkins_home:/var/jenkins_home
      - /var/run/docker.sock:/var/run/docker.sock
    user: root

volumes:
  jenkins_home:
```

Why mount `/var/run/docker.sock`? This allows Jenkins (running inside a container) to communicate with the host Docker daemon — so it can build and push images directly from within the pipeline.

17.9.1 Starting Jenkins

```
docker compose up -d
```

Then open: **`http://localhost:8080`**

17.10 10. Jenkins Configuration

17.10.1 Step 1 – Add Docker Hub Credentials

Navigate to:

Manage Jenkins → Credentials → System → Global credentials → Add Credentials

- **Kind:** Secret text
- **ID:** `dockerhub-token`
- **Secret:** Your Docker Hub access token

17.10.2 Step 2 – Create a Pipeline Job

1. Click **New Item**
 2. Select **Pipeline**
 3. Under **Pipeline Definition**, select **Pipeline script from SCM**
 4. Set **SCM** to **Git** and paste your GitHub repo URL
 5. Jenkins will automatically pick up the **Jenkinsfile** from the repo root
-

17.11 11. Webhook Integration

17.11.1 In GitHub

1. Go to your repo → **Settings** → **Webhooks** → **Add webhook**
2. Set the Payload URL to:
`http://<your-jenkins-ip>:8080/github-webhook/`
3. Content type: **application/json**
4. Trigger on: **Push events**

Now every **git** push will automatically trigger the Jenkins pipeline!

17.12 12. Execution Flow

Step 1: Developer pushes code to GitHub

↓

Step 2: GitHub sends a Webhook notification to Jenkins

↓

Step 3: Jenkins Pipeline runs:

- Clone code
- Build Docker image
- Login to Docker Hub
- Push image

↓

Step 4: Image is now live on Docker Hub

17.13 13. Key Concept: withCredentials

17.13.1 Problem

Hardcoding passwords in **Jenkinsfile** exposes secrets in version control.

17.13.2 Solution

```
withCredentials([string(credentialsId: 'dockerhub-token', variable: 'DOCKER_TOKEN')]) {  
    sh 'echo $DOCKER_TOKEN | docker login -u your-dockerhub-username --password-stdin'  
}
```

Jenkins:

- Fetches the secret from the encrypted credentials store
 - Injects it as an environment variable **temporarily**
 - Automatically masks it from all logs
 - Removes it from memory after the block completes
-

17.14 14. Key Commands in Pipeline

Command	Purpose
git	Clone the source code repository
sh	Execute a shell command on the agent
docker build	Build a Docker image from the Dockerfile
docker login	Authenticate with Docker Hub
docker push	Upload the image to Docker Hub

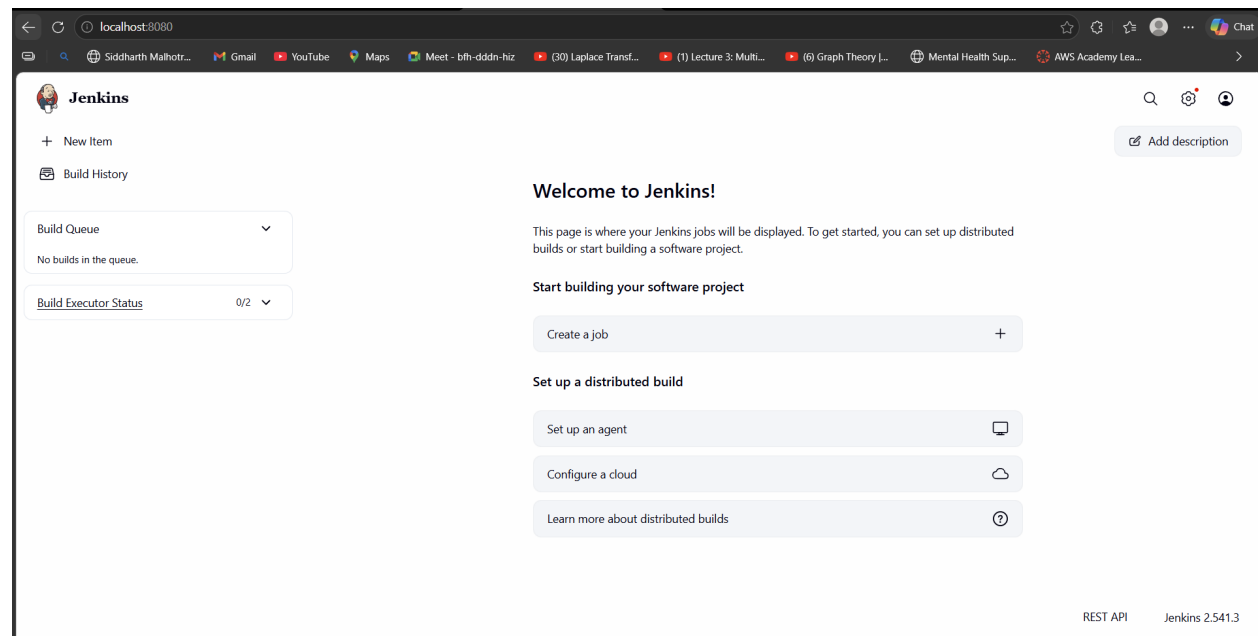
17.15 15. Observations & Screenshots

The following screenshots document the complete setup and execution of this CI/CD pipeline.

17.15.1 Jenkins Running via Docker Compose

```
PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-7> docker compose up
time="2026-04-18T14:51:54+05:30" level=warning msg="C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-7\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] up 2/2
✔Network lab-7_default Created 0.1s
✔Container jenkins Created 0.3s
PS C:\Users\Tushar Mangal\Documents\Containerization and DevOps\Lab\Lab-7\jenkins_lab-main> docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
691277f0a679   jenkins/jenkins:lts   "/usr/bin/tini -- /u..." 28 hours ago   Exited (137) 26 hours ago
12e373b7ce55   mysql:5.7          "docker-entrypoint.s..." 8 days ago    Up 9 minutes   3306/tcp, 3
3060/tcp      wordpress_db         "docker-entrypoint.s..." 5 days ago    Up 5 minutes   3306/tcp, 3
```

17.15.2 Jenkins Initial Setup & Dashboard



17.15.3 Adding Docker Hub Credentials in Jenkins

Update credential

×

Scope ?
Global (Jenkins, nodes, items, all child items, etc) ▼

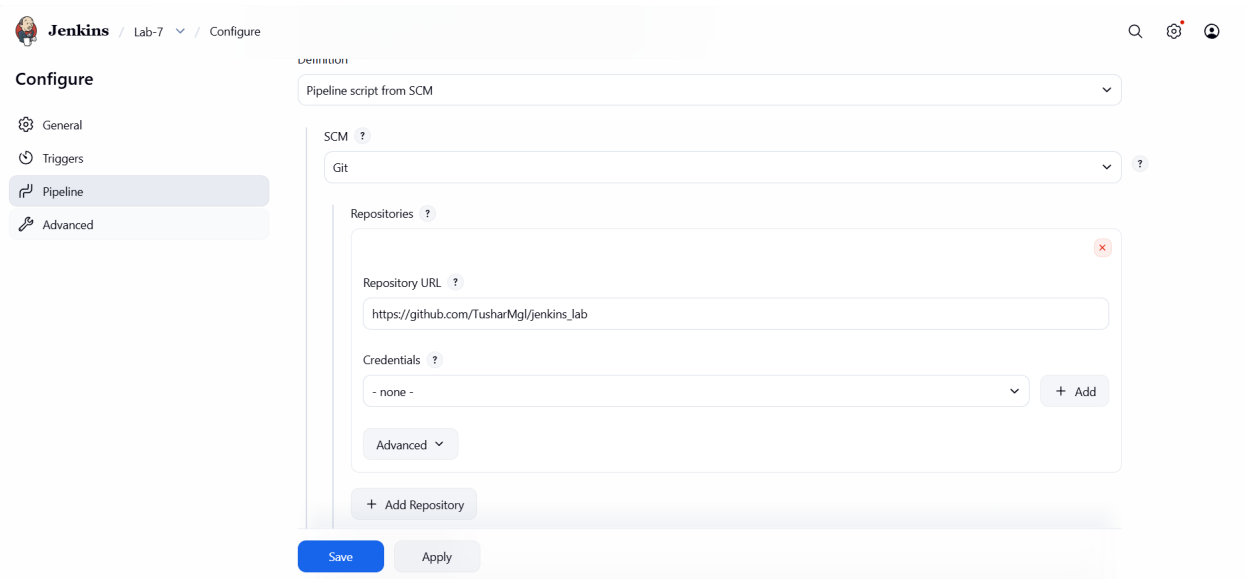
Secret
Concealed Change Password

ID ?
dockerhub-token

Description ?
Docker Hub Token

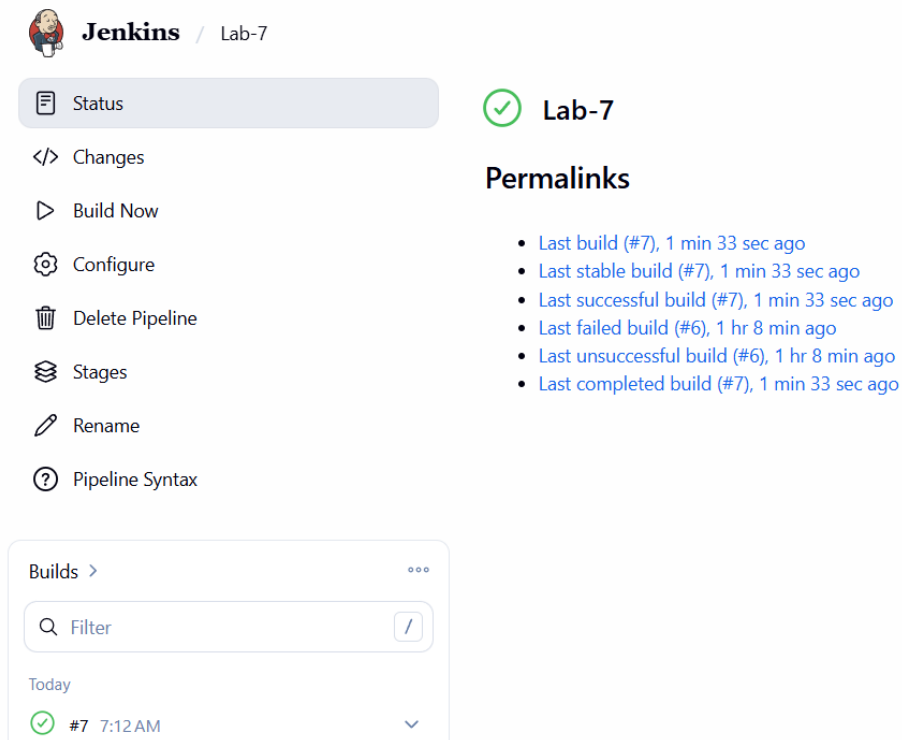
Save

17.15.4 Pipeline Job Configuration



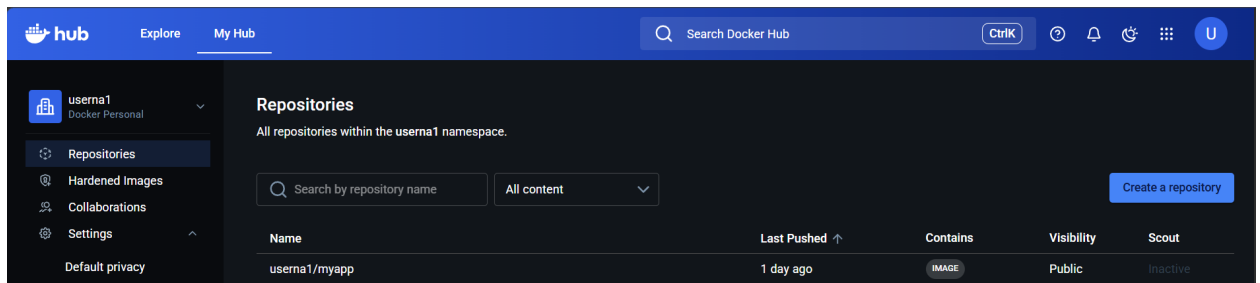
The screenshot shows the Jenkins configuration page for a pipeline job named 'Lab-7'. The left sidebar contains navigation links: General, Triggers, Pipeline (selected), and Advanced. The main configuration area is titled 'Configure' and includes a dropdown for 'Pipeline script from SCM'. Below this, the 'SCM' section is set to 'Git'. The 'Repositories' section contains a single repository with the URL 'https://github.com/TusharMgl/jenkins_lab' and credentials set to '- none -'. There are buttons for 'Add Repository', 'Save', and 'Apply' at the bottom.

17.15.5 Pipeline Build Execution



The screenshot shows the Jenkins interface for the 'Lab-7' pipeline. The left sidebar has a 'Status' tab selected, with links for Changes, Build Now, Configure, Delete Pipeline, Stages, Rename, and Pipeline Syntax. The main area shows a green checkmark icon and the text 'Lab-7'. Below this, the 'Permalinks' section lists several build links: 'Last build (#7), 1 min 33 sec ago', 'Last stable build (#7), 1 min 33 sec ago', 'Last successful build (#7), 1 min 33 sec ago', 'Last failed build (#6), 1 hr 8 min ago', 'Last unsuccessful build (#6), 1 hr 8 min ago', and 'Last completed build (#7), 1 min 33 sec ago'. At the bottom, the 'Builds' section shows a list of builds with a search filter and a table entry for build #7 at 7:12 AM, marked as successful with a green checkmark.

17.15.6 Image Successfully Pushed to Docker Hub



17.16

17.17 16. Result

Successfully created a complete automated CI/CD pipeline:

Component	Role
GitHub	Stores source code, triggers webhook on push
Jenkins	Automates build, login, and push stages
Docker	Provides consistent build environment
Docker Hub	Stores and distributes the built image

Every `git` push now **automatically** results in a new Docker image on Docker Hub — with **zero manual steps**.

17.18 Key Takeaways

1. **Jenkins automates** the entire pipeline — no manual intervention needed after setup.
2. **GitHub stores code** and acts as the trigger source via webhooks.
3. **Docker ensures consistency** — the same image runs anywhere.
4. **withCredentials** safely handles secrets without exposing them in logs or code.
5. **Pipeline as Code** (Jenkinsfile) keeps your CI/CD logic version-controlled alongside your app.

18 Containerization-and-DevOps

18.1 Experiment 10: Working with SonarQube

18.1.0.1 Introduction

18.1.0.2 What is SonarQube? SonarQube is an open-source platform for continuous inspection of code quality. It performs automatic reviews with static analysis to detect bugs, code smells, and security vulnerabilities.

18.1.0.3 How SonarQube Solves the Problem:

- **Continuous Inspection:** Scans code with every commit, providing immediate feedback.
- **Quality Gates:** Defines pass/fail criteria for code quality.

- **Technical Debt Quantification:** Measures effort needed to fix issues.
 - **Multi-language Support:** Supports 20+ programming languages.
 - **Visual Analytics:** Dashboard showing code quality metrics and trends.
-

18.1.0.4 Hands-On

Step-1:- Create a docker-compose.yml

`nano docker-compose.yml`

Paste This:

```
version: '3.8'

services:
  sonar-db:
    image: postgres:13
    container_name: sonar-db
    environment:
      POSTGRES_USER: sonar
      POSTGRES_PASSWORD: sonar
      POSTGRES_DB: sonarqube
    volumes:
      - sonar-db-data:/var/lib/postgresql/data
    networks:
      - sonarqube-lab

  sonarqube:
    image: sonarqube:lts-community
    container_name: sonarqube
    ports:
      - "9000:9000"
    environment:
      SONAR_JDBC_URL: jdbc:postgresql://sonar-db:5432/sonarqube
      SONAR_JDBC_USERNAME: sonar
      SONAR_JDBC_PASSWORD: sonar
    volumes:
      - sonar-data:/opt/sonarqube/data
      - sonar-extensions:/opt/sonarqube/extensions
    depends_on:
      - sonar-db
    networks:
      - sonarqube-lab

volumes:
  sonar-db-data:
  sonar-data:
  sonar-extensions:

networks:
  sonarqube-lab:
    driver: bridge
```

```

tushar_mangal@LAPTOP-1DD  x  +  v
GNU nano 7.2                                docker-compose.yml *
version: '3.8'

services:
  sonar-db:
    image: postgres:13
    container_name: sonar-db
    environment:
      POSTGRES_USER: sonar
      POSTGRES_PASSWORD: sonar
      POSTGRES_DB: sonarqube
    volumes:
      - sonar-db-data:/var/lib/postgresql/data
    networks:
      - sonarqube-lab

  sonarqube:
    image: sonarqube:lts-community
    container_name: sonarqube
    ports:
      - "9000:9000"
    environment:
      SONAR_JDBC_URL: jdbc:postgresql://sonar-db:5432/sonarqube
      SONAR_JDBC_USERNAME: sonar
      SONAR_JDBC_PASSWORD: sonar
    volumes:
      - sonar-data:/opt/sonarqube/data

```

Step-2:- Start Compose

docker-compose up -d

```

tushar_mangal@LAPTOP-1DDBCFLP:/mnt/c/Users/Tushar Mangal/Documents/Containerization and DevOps/Lab/Lab-10$ docker-compose up -d
WARN[0000] /mnt/c/Users/Tushar Mangal/Documents/Containerization and DevOps/Lab/Lab-10/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] up 34/34
✔Image postgres:13                               Pulled                               46.7s
✔Image sonarqube:lts-community                    Pulled                               66.0s
✔Network lab-10_sonarqube-lab                     Created                                0.2s
✔Volume lab-10_sonar-db-data                       Created                                0.0s
✔Volume lab-10_sonar-data                         Created                                0.0s
✔Volume lab-10_sonar-extensions                   Created                                0.0s
✔Container sonar-db                               Created                                1.1s
✔Container sonarqube                              Created                                0.3s

```

Step-3:- Verify SonarQube is running via Logs

docker-compose logs -f sonarqube

```

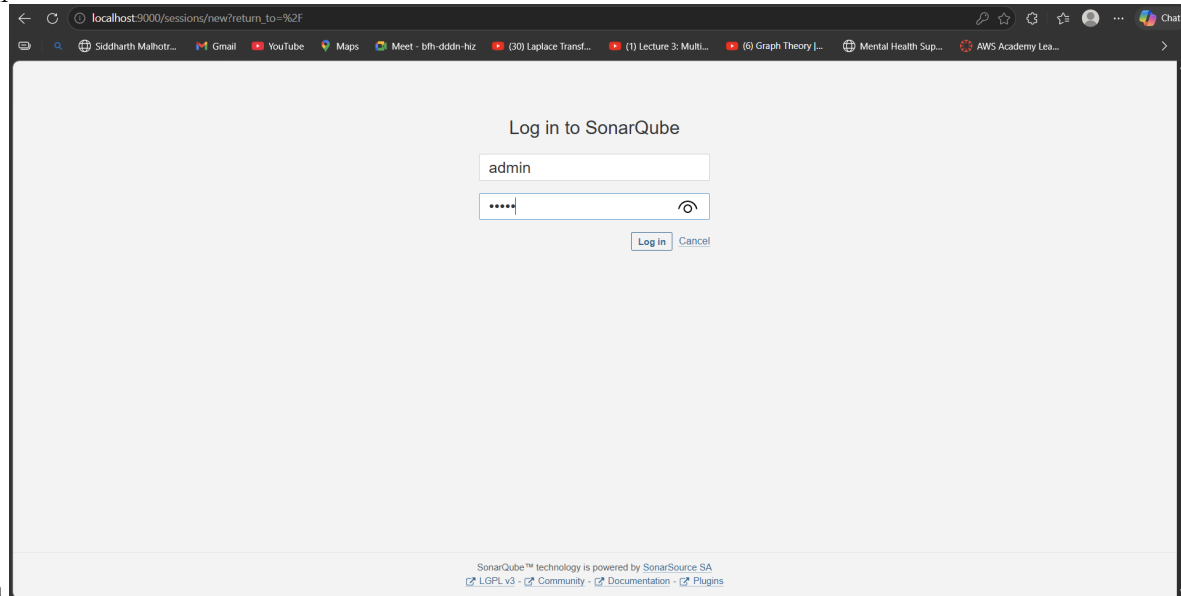
tushar_mangal@LAPTOP-1DDBCFLP:/mnt/c/Users/Tushar Mangal/Documents/Containerization and DevOps/Lab/Lab-10$ docker-compose logs -f sonarqube
WARN[0000] /mnt/c/Users/Tushar Mangal/Documents/Containerization and DevOps/Lab/Lab-10/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
sonarqube | 2026.04.25 06:58:56 INFO app[[o.s.a.AppFileSystem] Cleaning or creating temp directory /opt/sonarqube/temp
sonarqube | 2026.04.25 06:58:56 INFO app[[o.s.a.es.EsSettings] Elasticsearch listening on [HTTP: 127.0.0.1:9001, TCP: 127.0.0.1:38845]
sonarqube | 2026.04.25 06:58:56 INFO app[[o.s.a.ProcessLauncherImpl] Launch process[ELASTICSEARCH] from [/opt/sonarqube/elasticsearch]: /opt/sonarqube/elasticsearch/bin/elasticsearch
sonarqube | 2026.04.25 06:58:56 INFO app[[o.s.a.SchedulerImpl] Waiting for Elasticsearch to be up and running
sonarqube | 2026.04.25 06:58:59 INFO es[[o.e.n.Node] version[7.17.15], pid[26], build[default/tar/0b8ecfb4378335f4689c4223d1f115f16bef3ba/2023-11-10T22:03:46.987399016Z], OS[Linux/6.6.87.2-microsoft-standard-WSL2/amd64], JVM[Eclipse Adoptium/OpenJDK 64-Bit Server VM/17.0.16/17.0.16+8]
sonarqube | 2026.04.25 06:58:59 INFO es[[o.e.n.Node] JVM home [/opt/java/openjdk]
sonarqube | 2026.04.25 06:58:59 INFO es[[o.e.n.Node] JVM arguments [-XX:+UseG1GC, -Djava.io.tmpdir=/opt/sonarqube/temp, -XX:ErrorFile=/opt/sonarqube/logs/es_hs_err_pid%p.log, -Des.networkaddress.cache.ttl=60, -Des.networkaddress.cache.negative.ttl=10, -XX:+AlwaysPreTouch, -Xss1m, -Djava.awt.headless=true, -Dfile.encoding=UTF-8, -Djna.nosys=true, -Djna.tmpdir=/opt/sonarqube/temp, -XX:-OmitStackTraceInFastThrow, -Dio.netty.noUnsafe=true, -Dio.netty.noKeySetOptimization=true, -Dio.netty.recycler.maxCapacityPerThread=0, -Dio.netty allocator.numDirectArenas=0, -Dlog4j.shutdownHookEnabled=false, -Dlog4j2.disable.jmx=true, -Dlog4j2.formatMsgNoLookups=true, -Djava.locale.providers=COMPAT, -Dcom.redhat.fips=false, -Des.enforce.bootstrap.checks=true, -Xmx512m, -Xms512m, -XX:MaxDirectMemorySize=256m, -XX:+HeapDumpOnOutOfMemoryError, -Des.path.home=/opt/sonarqube/elasticsearch, -Des.path.conf=/opt/sonarqube/temp/conf/es, -Des.distribution.flavor=default, -Des.distribution.type=tar, -Des.bundled.jdk=false]
sonarqube | 2026.04.25 06:59:00 INFO es[[o.e.p.PluginsService] loaded module [analysis-common]
sonarqube | 2026.04.25 06:59:00 INFO es[[o.e.p.PluginsService] loaded module [lang-painless]
sonarqube | 2026.04.25 06:59:00 INFO es[[o.e.p.PluginsService] loaded module [parent-join]

```

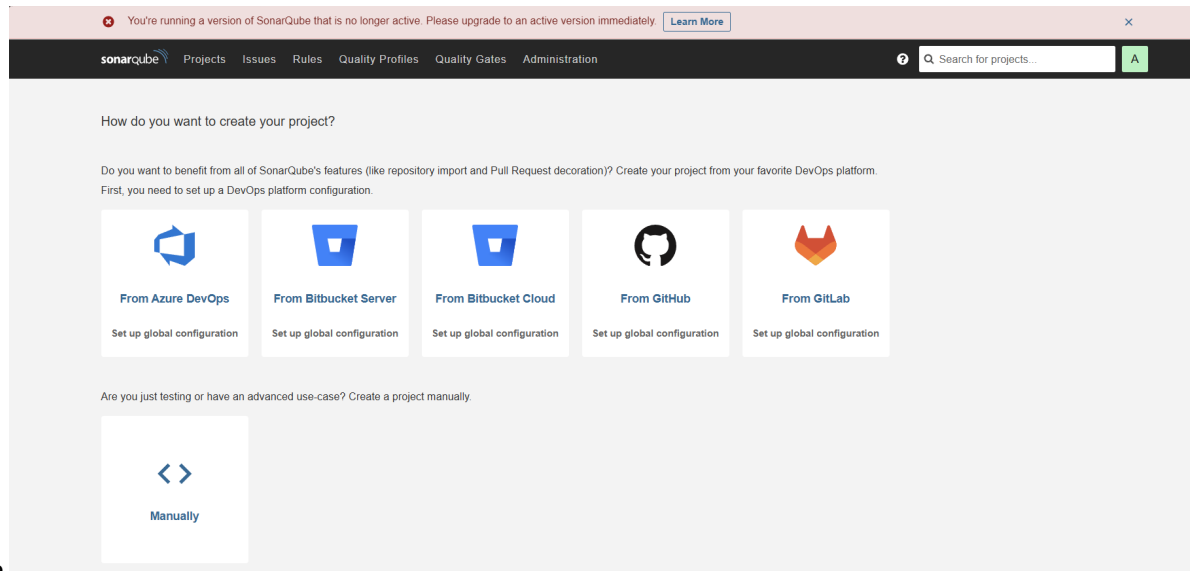
Step-4:- Login SonarQube

http://localhost:9001

- username: admin



- password: admin



Step-5:- HomePage

Step-6:- Create Sample Java App

```
mkdir -p sample-java-app/src/main/java/com/example
cd sample-java-app
```

Step-7:- Create Calculator.java

```
nano src/main/java/com/example/Calculator.java

package com.example;

public class Calculator {

    public int divide(int a, int b) {
        return a / b;
    }
}
```



```

public int add(int a, int b) {
    int result = a + b;
    int unused = 100;
    return result;
}

public String getUser(String userId) {
    String query = "SELECT * FROM users WHERE id = " + userId;
    return query;
}

public int multiply(int a, int b) {
    int result = 0;
    for (int i = 0; i < b; i++) {
        result = result + a;
    }
    return result;
}

public int multiplyAlt(int a, int b) {
    int result = 0;
    for (int i = 0; i < b; i++) {
        result = result + a;
    }
    return result;
}

public String getName(String name) {
    return name.toUpperCase();
}

public void riskyOperation() {
    try {
        int x = 10 / 0;
    } catch (Exception e) {
    }
}
}

```

```
tushar_mangal@LAPTOP-1DD  x  +  v  -  □  x
GNU nano 7.2  src/main/java/com/example/Calculator.java
package com.example;

public class Calculator {

    public int divide(int a, int b) {
        return a / b;
    }

    public int add(int a, int b) {
        int result = a + b;
        int unused = 100;
        return result;
    }

    public String getUser(String userId) {
        String query = "SELECT * FROM users WHERE id = " + userId;
        return query;
    }

    public int multiply(int a, int b) {
        int result = 0;
        for (int i = 0; i < b; i++) {
            result = result + a;
        }
        return result;
    }
}
```

Step-8:- Create pom.xml

nano pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>
  <artifactId>sample-app</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <sonar.projectKey>sample-java-app</sonar.projectKey>
    <sonar.host.url>http://localhost:9000</sonar.host.url>
    <sonar.login>YOUR_TOKEN</sonar.login>
  </properties>

  <build>
    <plugins>
      <plugin>
        <groupId>org.sonarsource.scanner.maven</groupId>
        <artifactId>sonar-maven-plugin</artifactId>
        <version>3.9.1.2184</version>
      </plugin>
    </plugins>
  </build>
</project>
```

```

GNU nano 7.2 pom.xml
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>

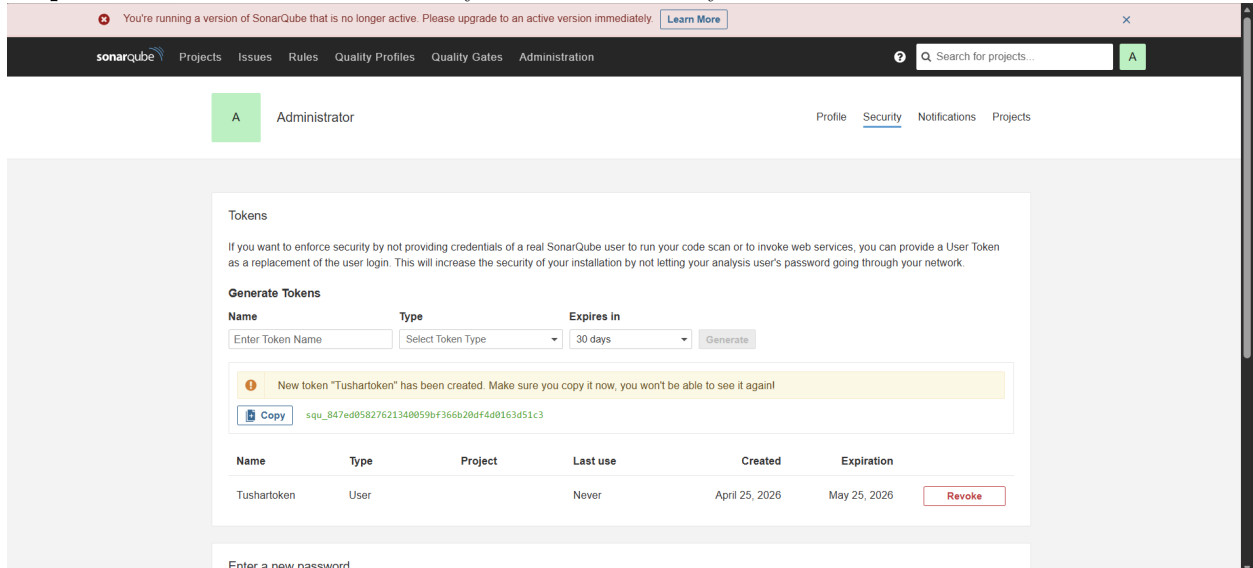
  <groupId>com.example</groupId>
  <artifactId>sample-app</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <sonar.projectKey>sample-java-app</sonar.projectKey>
    <sonar.host.url>http://localhost:9000</sonar.host.url>
    <sonar.login>YOUR_TOKEN</sonar.login>
  </properties>

  <build>
    <plugins>
      <plugin>
        <groupId>org.sonarsource.scanner.maven</groupId>
        <artifactId>sonar-maven-plugin</artifactId>
        <version>3.9.1.2184</version>
      </plugin>
    </plugins>
  </build>
</project>

```

Step-9:- Create Tokens Profile -> My Account -> Security -> TokenName: scanner-token -> Generate



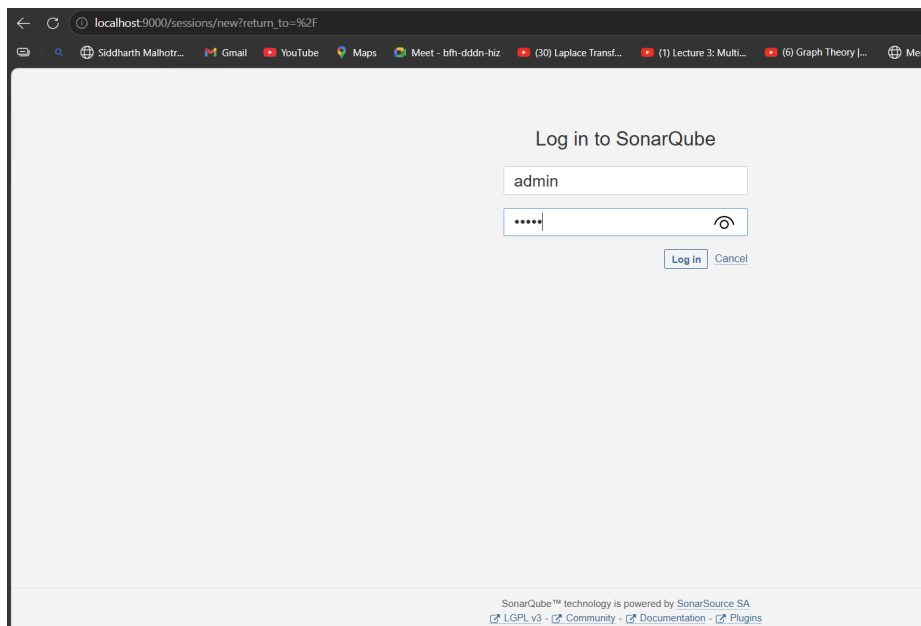
Step-10:- Run Scanner

`mvn sonar:sonar -Dsonar.login=YOUR_TOKEN`

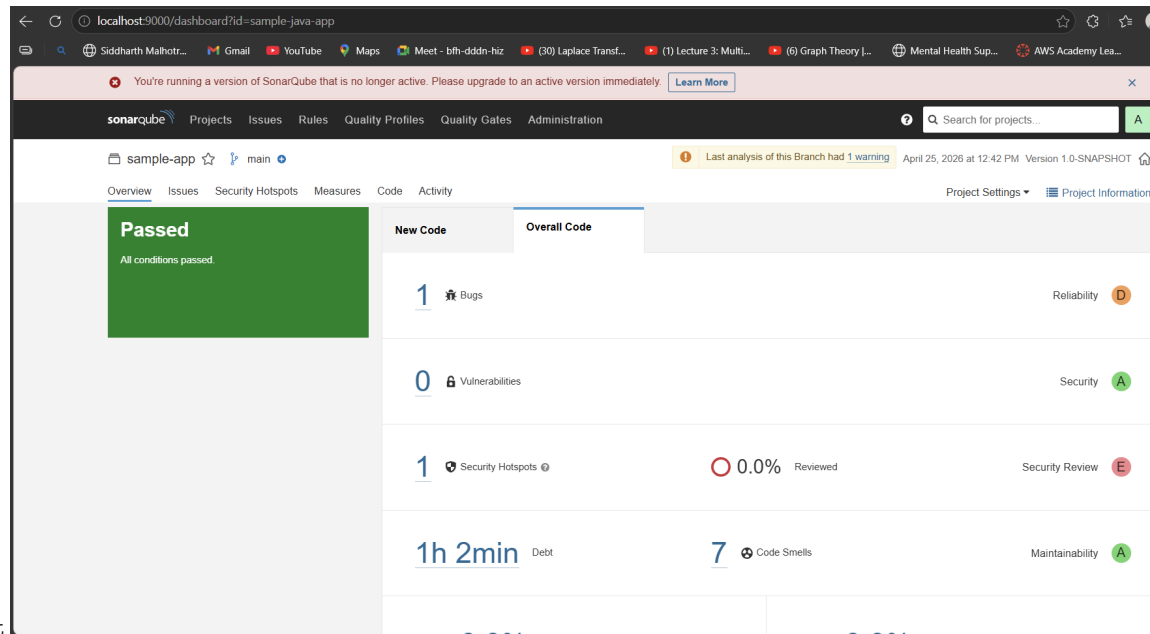
```
tushar_mangal@LAPTOP-IDDBCFPL: /mnt/c/Users/Tushar Mangal/Documents/Containerization and DevOps/Lab/Lab-10/sample-java-ap
p$ mvn sonar:sonar -Dsonar.login=squ_847ed05827621340059bf366b20df4d0163d51c3
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.example:sample-app >-----
[INFO] Building sample-app 1.0-SNAPSHOT
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- sonar-maven-plugin:3.9.1.2184:sonar (default-cli) @ sample-app ---
[INFO] User cache: /home/tushar_mangal/.sonar/cache
[INFO] SonarQube version: 9.9.8.100196
[INFO] Default locale: "en", source code encoding: "UTF-8" (analysis is platform dependent)
[INFO] Load global settings
[INFO] Load global settings (done) | time=192ms
[INFO] Server id: 74C15348-AZ3Db49ZBntJ7P7afIFZ
[INFO] User cache: /home/tushar_mangal/.sonar/cache
[INFO] Load/download plugins
[INFO] Load plugins index
[INFO] Load plugins index (done) | time=118ms
[INFO] Load/download plugins (done) | time=4240ms
[INFO] Process project properties
[INFO] Process project properties (done) | time=28ms
[INFO] Execute project builders
```

```
[WARNING] Thread[#50,JGit-FileStoreAttributeReader-1,5,main]: got smaller file times
tushar Mangal/Documents/Containerization and DevOps/Lab/.git: 2026-04-25T07:12:27Z < 2
ing measurement at resolution PT0.0154685S.
[INFO] SCM Publisher 0/2 source files have been analyzed (done) | time=673ms
[WARNING] Missing blame information for the following files:
[WARNING] * src/main/java/com/example/Calculator.java
[WARNING] * pom.xml
[WARNING] This may lead to missing/broken features in SonarQube
[INFO] CPD Executor Calculating CPD for 1 file
[INFO] CPD Executor CPD calculation finished (done) | time=14ms
[INFO] Analysis report generated in 238ms, dir size=127.8 kB
[INFO] Analysis report compressed in 1056ms, zip size=19.2 kB
[INFO] Analysis report uploaded in 429ms
[INFO] ANALYSIS SUCCESSFUL, you can find the results at: http://localhost:9000/dashbo
[INFO] Note that you will be able to access the updated dashboard once the server has
report
[INFO] More about the report processing at http://localhost:9000/api/ce/task?id=AZ3D
[INFO] Analysis total time: 13.836 s
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 20.331 s
[INFO] Finished at: 2026-04-25T07:12:30Z
[INFO]
```

Step-11:- Scroll down to view Build Success



Step-12:- On Browser test will be Listed



Step-13:- View Report

19 Containerization-and-DevOps

19.1 Experiment 11: Orchestration via Docker Swarm

19.1.0.1 Introduction

19.1.0.2 The Progression Path

docker run → Docker Compose → Docker Swarm → Kubernetes

Single container	Multi-container (single host)	Orchestration (basic)	Advanced orchestration
------------------	----------------------------------	--------------------------	---------------------------

This experiment focuses on: Moving from Compose → Swarm

19.1.0.3 Hands-On

Step-1:- Create a docker-compose.yml

```
nano docker-compose.yml
```

Paste This:

```
version: '3.9'
```

```
services:
```

```
  db:
```

```

image: mysql:5.7
restart: always
environment:
  MYSQL_ROOT_PASSWORD: rootpass
  MYSQL_DATABASE: wordpress
  MYSQL_USER: wpuser
  MYSQL_PASSWORD: wppass
volumes:
  - db_data:/var/lib/mysql

```

```

wordpress:
  image: wordpress:latest
  depends_on:
    - db
  ports:
    - "8080:80"
  restart: always
  environment:
    WORDPRESS_DB_HOST: db:3306
    WORDPRESS_DB_USER: wpuser
    WORDPRESS_DB_PASSWORD: wppass
    WORDPRESS_DB_NAME: wordpress
  volumes:
    - wp_data:/var/www/html

```

```

volumes:
  db_data:
  wp_data:

```

The screenshot shows a terminal window with the title 'tushar_mangal@LAPTOP-1DD'. The editor is GNU nano 7.2, editing 'docker-compose.yml'. The file content is as follows:

```

version: '3.9'

services:
  db:
    image: mysql:5.7
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: rootpass
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wpuser
      MYSQL_PASSWORD: wppass
    volumes:
      - db_data:/var/lib/mysql

  wordpress:
    image: wordpress:latest
    depends_on:
      - db
    ports:
      - "8080:80"
    restart: always
    environment:
      WORDPRESS_DB_HOST: db:3306
      WORDPRESS_DB_USER: wpuser
      WORDPRESS_DB_PASSWORD: wppass
      WORDPRESS_DB_NAME: wordpress

```

At the bottom of the terminal, there is a status bar with various keyboard shortcuts like ^G Help, ^O Write Out, ^W Where Is, ^K Cut, ^T Execute, ^C Location, M-U Undo, M-A Set Mark, ^X Exit, ^R Read File, ^\ Replace, ^U Paste, ^J Justify, ^_ Go To Line, M-E Redo, and M-6 Copy. A message '[Read 32 lines]' is also visible.

Step-2:- Initialize Swarm

```
docker swarm init
```

```
tushar_mangal@LAPTOP-1DDBCFLP:/mnt/c/Users/Tushar Mangal/Documents/Containerization and DevOps/Lab/Lab-11$ docker swarm
init
Swarm initialized: current node (lg5xjls2gvceqkvi7lxdeywj5) is now a manager.

To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-2vuhfoic12jlwnav4gelksw0zu7g64kxn57vo335rbixl20k3y-1rr5m0o10o3ubavvj799qg3vn 192.168.65.3:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.
```

Step-3:- Verify Swarm is Active

docker node ls

```
tushar_mangal@LAPTOP-1DDBCFLP:/mnt/c/Users/Tushar Mangal/Documents/Containerization and DevOps/Lab/Lab-11$ docker node ls
ID                HOSTNAME          STATUS    AVAILABILITY    MANAGER STATUS  ENGINE VERSION
lg5xjls2gvceqkvi7lxdeywj5 *  docker-desktop  Ready    Active           Leader           29.2.1
```

Step-4:- Deploy as a stack

docker stack deploy -c docker-compose.yml wpstack

```
tushar_mangal@LAPTOP-1DDBCFLP:/mnt/c/Users/Tushar Mangal/Documents/Containerization and DevOps/Lab/Lab-11$ docker stack
deploy -c docker-compose.yml wpstack
Ignoring unsupported options: restart

Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating network wpstack_default
Creating service wpstack_db
Creating service wpstack_wordpress
```

```
tushar_mangal@LAPTOP-1DDBCFLP:/mnt/c/Users/Tushar Mangal/Documents/Containerization and DevOps/Lab/Lab-11$ docker stack
e ls
ID                NAME                MODE                REPLICAS    IMAGE                PORTS
wcfbhu3skuc9      wpstack_db          replicated          1/1         mysql:5.7            *
yo5b4ru2vtf       wpstack_wordpress   replicated          1/1         wordpress:latest     *:8080->80/tcp
```

Step-5:- Verify by listing Services

Step-6:- Details for container service

docker service ps wpstack_wordpress

```
tushar_mangal@LAPTOP-1DDBCFLP:/mnt/c/Users/Tushar Mangal/Documents/Containerization and DevOps/Lab/Lab-11$ docker servic
e ps wpstack_wordpress
ID                PORTS                NAME                IMAGE                NODE                DESIRED STATE    CURRENT STATE    ERROR
f64n7jttlxc7g    wpstack_wordpress.1  wordpress:latest    docker-desktop      Running            Running 13 seconds ago
```

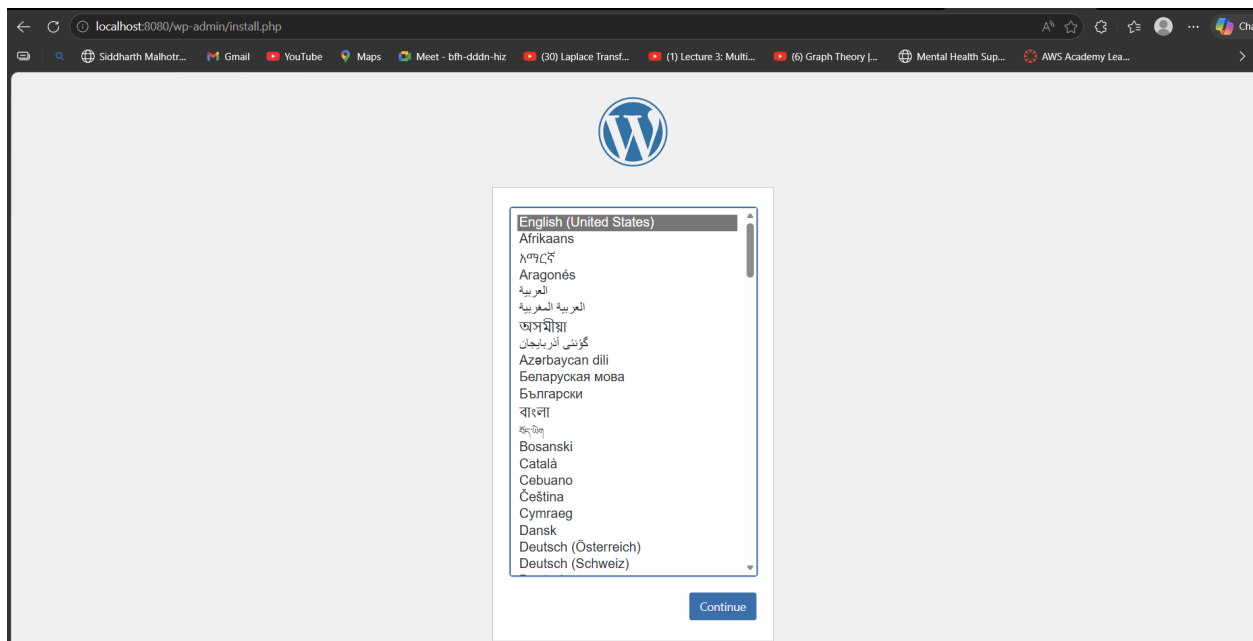
Step-7:- List Containers

docker ps

```
tushar_mangal@LAPTOP-1DDBCFLP:/mnt/c/Users/Tushar Mangal/Documents/Containerization and DevOps/Lab/Lab-11$ docker ps
CONTAINER ID    IMAGE                COMMAND              CREATED          STATUS              NAMES                PORTS
7c4250c498f8    wordpress:latest     "docker-entrypoint.s..."  21 seconds ago  Up 20 seconds      wpstack_wordpress.1  80/tcp
bi3c0km9uux     mysql:5.7            "docker-entrypoint.s..."  26 seconds ago  Up 25 seconds      wpstack_db.1.w32pfajxaeuxjjqjw9uv3  3306/tcp
f8e585aba56c    jenkins/jenkins:lts "/usr/bin/tini -- /u..."  12 days ago     Up 11 minutes      jenkins              0.0.0.0:50000->50000/tcp, [::]:50000->50000/tcp, 0.0.0.0:8081->8080/tcp, [::]:8081->8080/tcp
12e373b7ce55    mysql:5.7            "docker-entrypoint.s..."  2 weeks ago     Up 11 minutes      wordpress_db         3306/tcp
7bbd239ea7c8    node:18-alpine       "docker-entrypoint.s..."  2 weeks ago     Restarting (0) 21 seconds ago  webapp
```

Step-8:- Verify application on Containers

http://localhost:8080/



Step-9:- Scale application Containers

`docker service scale wpstack_wordpress=3`

```
tushar_mangal@LAPTOP-1DDBCFLP:/mnt/c/Users/Tushar Mangal/Documents/Containerization and DevOps/Lab/Lab-11$ docker service scale wpstack_wordpress=3
wpstack_wordpress scaled to 3
overall progress: 3 out of 3 tasks
1/3: running [=====>]
2/3: running [=====>]
3/3: running [=====>]
verify: Service wpstack_wordpress converged
```

Step-10:- Verify Scalling

`docker service ls`

```
tushar_mangal@LAPTOP-1DDBCFLP:/mnt/c/Users/Tushar Mangal/Documents/Containerization and DevOps/Lab/Lab-11$ docker service ls
ID                NAME                MODE                REPLICAS            IMAGE                PORTS
wcfbhu3skuc9      wpstack_db           replicated          1/1                 mysql:5.7
yo5b4ru2uvtf      wpstack_wordpress    replicated          3/3                 wordpress:latest     *:8080->80/tcp
```

Step-11:- List Wordpress Containers

`docker ps | grep wordpress`

```
tushar_mangal@LAPTOP-1DDBCFLP:/mnt/c/Users/Tushar Mangal/Documents/Containerization and DevOps/Lab/Lab-11$ docker ps | grep wordpress
50444b113059      wordpress:latest     "docker-entrypoint.s..." 18 seconds ago      Up 17 seconds      80/tcp
q67h0b33mjum876      wordpress:latest     "docker-entrypoint.s..." 18 seconds ago      Up 17 seconds      80/tcp
64658cb019fe      wordpress:latest     "docker-entrypoint.s..." 18 seconds ago      Up 17 seconds      80/tcp
nxlt5bkzpr2zs2q2      wordpress:latest     "docker-entrypoint.s..." About a minute ago   Up About a minute   80/tcp
7c4250c498f8      wordpress:latest     "docker-entrypoint.s..." About a minute ago   Up About a minute   80/tcp
0c7gbi3c0km9uux      mysql:5.7            "docker-entrypoint.s..." 2 weeks ago         Up 12 minutes      3306/tcp
12e373b7ce55      mysql:5.7            "docker-entrypoint.s..." 2 weeks ago         Up 12 minutes      3306/tcp
```

Step-12:- Kill any Container

`docker kill <container-id>`

```
tushar_mangal@LAPTOP-1DDBCFLP:/mnt/c/Users/Tushar Mangal/Documents/Containerization and DevOps/Lab/Lab-11$ docker kill 50444b113059
50444b113059
```

Step-13:- Watch Swarm Recreating Contrainer


```
docker service ps wpstack_wordpress
```

```
tushar_mangal@LAPTOP-1DDBCFLP:/mnt/c/Users/Tushar Mangal/Documents/Containerization and DevOps/Lab/Lab-11$ docker service ps wpstack_wordpress
```

ID	NAME	IMAGE	NODE	DESIRED STATE	CURRENT STATE
f64n7jttlx0c	wpstack_wordpress.1	wordpress:latest	docker-desktop	Running	Running 2 minutes ago
ibzh08ccgenx	wpstack_wordpress.2	wordpress:latest	docker-desktop	Running	Running about a minute ago
ar0l9c6u5age	wpstack_wordpress.3	wordpress:latest	docker-desktop	Running	Running less than a second ago

Step-14:- Verify Container running

```
docker ps | grep wordpress
```

```
tushar_mangal@LAPTOP-1DDBCFLP:/mnt/c/Users/Tushar Mangal/Documents/Containerization and DevOps/Lab/Lab-11$ docker ps | grep wordpress
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
62dc0b685c5e	wordpress:latest	"docker-entrypoint.s..."	18 seconds ago	Up 12 seconds	80/tcp
wpstack_wordpress.3.ar0l9c6u5a					
64658cb019fe	wordpress:latest	"docker-entrypoint.s..."	About a minute ago	Up About a minute	80/tcp
wpstack_wordpress.2.ibzh08ccge					
7c4250c498f8	wordpress:latest	"docker-entrypoint.s..."	2 minutes ago	Up 2 minutes	80/tcp
wpstack_wordpress.1.f64n7jttlx					
12e373b7ce55	mysql:5.7	"docker-entrypoint.s..."	2 weeks ago	Up 13 minutes	3306/tcp
wordpress_db					

Step-15:- Remove Stack

```
docker stack rm wpstack
```

```
tushar_mangal@LAPTOP-1DDBCFLP:/mnt/c/Users/Tushar Mangal/Documents/Containerization and DevOps/Lab/Lab-11$ docker stack rm wpstack
Removing service wpstack_db
Removing service wpstack_wordpress
Removing network wpstack_default
```

Step-16:- Verify Service Removal

```
docker service ls
```

```
docker ps
```

```
tushar_mangal@LAPTOP-1DDBCFLP:/mnt/c/Users/Tushar Mangal/Documents/Containerization and DevOps/Lab/Lab-11$ docker service ls
```

ID	NAME	MODE	REPLICAS	IMAGE	PORTS
f8e585aba56c	jenkins/jenkins:lts		12 days ago	Up 14 minutes	0.0.0.0:5000
0->50000/tcp, [::]:50000->50000/tcp,					
12e373b7ce55	mysql:5.7		2 weeks ago	Up 14 minutes	3306/tcp, 3306/tcp
060/tcp					
7bbd239ea7c8	node:18-alpine		2 weeks ago	Restarting (0) 46 seconds ago	webapp

19.1.0.4 Conclusion

19.1.0.5 Summary

You started with	You can now do
Single container (<code>docker run</code>)	Multi-container (Compose)
Manual scaling	One-command scaling (<code>scale</code>)
Manual recovery	Automatic self-healing
Single host	Multi-host cluster ready

19.1.0.6 Final Takeaway

Compose defines the application. Swarm runs it reliably.

19.1.0.7 Quick Reference Card

```
# Initialize Swarm
docker swarm init

# Deploy stack
docker stack deploy -c docker-compose.yml <stack-name>

# List services
docker service ls

# Scale service
docker service scale <stack-name_service-name>=<replicas>

# See service tasks
docker service ps <service-name>

# Remove stack
docker stack rm <stack-name>

# Leave Swarm (if needed)
docker swarm leave --force
```

20 Containerization-and-DevOps

20.1 Experiment 12: Kubernetes

20.1.0.1 Introduction

20.1.0.2 Why Kubernetes over Docker Swarm?

Reason	Explanation
Industry standard	Most companies use Kubernetes
Powerful scheduling	Automatically decides where to run your app
Large ecosystem	Many tools and plugins available
Cloud-native support	Works on AWS, Google Cloud, Azure, etc.

20.1.0.3 Core Kubernetes Concepts (Simple Explanation)

Docker Concept	Kubernetes Equivalent	What it means
Container	Pod	A pod is a group of one or more containers. Smallest unit in K8s.

Docker Concept	Kubernetes Equivalent	What it means
Compose service	Deployment	Describes how your app should run (e.g., 2 copies, which image to use)
Load balancing	Service	Exposes your app to the outside world or other pods
Scaling	ReplicaSet	Ensures a certain number of pod copies are always running

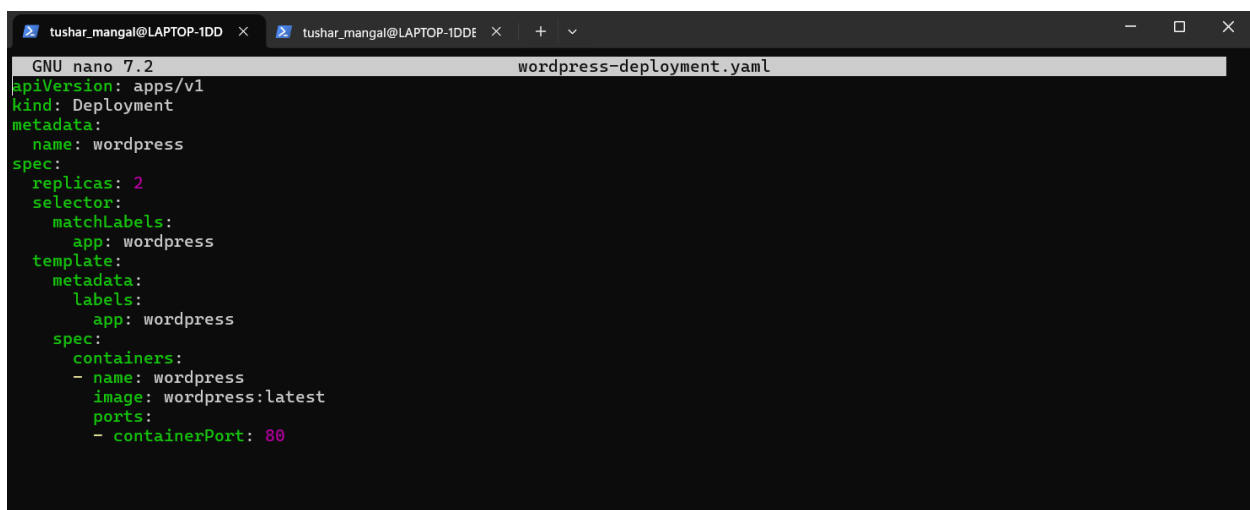
20.1.0.4 Hands-On

Step-1:- Create a wordpress-deployment.yaml

nano wordpress-deployment.yaml

Paste This:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: wordpress
spec:
  replicas: 2
  selector:
    matchLabels:
      app: wordpress
  template:
    metadata:
      labels:
        app: wordpress
    spec:
      containers:
      - name: wordpress
        image: wordpress:latest
        ports:
        - containerPort: 80
```



The screenshot shows a terminal window with two tabs. The active tab is titled 'tushar_mangal@LAPTOP-1DD' and contains the GNU nano 7.2 editor. The editor is editing a file named 'wordpress-deployment.yaml'. The content of the file is the same YAML configuration as shown in the previous block, with syntax highlighting applied to various fields like 'apiVersion', 'kind', 'spec', and 'replicas'.

Step-2:- Apply Deployment

kubectl apply -f wordpress-deployment.yaml

```
tushar_mangal@LAPTOP-1DDBCFLP:/mnt/c/Users/Tushar Mangal/Documents/Containerization and DevOps/Lab/Lab-12$ kubectl apply
-f wordpress-deployment.yaml
deployment.apps/wordpress created
```

Step-3:- Create wordpress-service.yaml

```
nano wordpress-service.yaml
```

Paste this:-

```
apiVersion: v1
kind: Service
metadata:
  name: wordpress-service
spec:
  type: NodePort
  selector:
    app: wordpress
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30007
```

```
GNU nano 7.2 wordpress-service.yaml
apiVersion: v1
kind: Service
metadata:
  name: wordpress-service
spec:
  type: NodePort
  selector:
    app: wordpress
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30007
```

Step-4:- Deploy Service

```
kubectl apply -f wordpress-service.yaml
```

```
tushar_mangal@LAPTOP-1DDBCFLP:/mnt/c/Users/Tushar Mangal/Documents/Containerization and DevOps/Lab/Lab-12$ kubectl apply
-f wordpress-service.yaml
service/wordpress-service created
```

Step-5:- Get pods

```
kubectl get pods
```

```
tushar_mangal@LAPTOP-1DDBCFLP:/mnt/c/Users/Tushar Mangal/Documents/Containerization and DevOps/Lab/Lab-12$ kubectl get p
ods
NAME                                READY   STATUS    RESTARTS   AGE
wordpress-848c87c44f-djdnf         1/1     Running   0           83s
wordpress-848c87c44f-srbmb         1/1     Running   0           83s
```

Step-6:- Check service

```
kubectl get svc
```

```
tushar_mangal@LAPTOP-1DDBCFLP:/mnt/c/Users/Tushar Mangal/Documents/Containerization and DevOps/Lab/Lab-12$ kubectl get s
vc
NAME              TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
kubernetes        ClusterIP   10.96.0.1    <none>         443/TCP          2m26s
wordpress-service NodePort    10.111.48.226 <none>         80:30007/TCP     19s
```

http://localhost:8080/



```
tushar_mangal@LAPTOP-1DDBCFLLP: /mnt/c/Users/Tushar Mangal/Documents/Containerization and DevOps/Lab/Lab-12$ kubectl scale deployment wordpress --replicas=4
deployment.apps/wordpress scaled
```

```
tushar_mangal@LAPTOP-1DDBCFLP: /mnt/c/Users/Tushar Mangal/Documents/Containerization and DevOps/Lab/Lab-12$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
wordpress-848c87c44f-7mzmk	1/1	Running	0	6s
wordpress-848c87c44f-djdnp	1/1	Running	0	2m15s
wordpress-848c87c44f-knb27	0/1	ContainerCreating	0	6s
wordpress-848c87c44f-srbmb	1/1	Running	0	2m15s

```
tushar_mangal@LAPTOP-1DDBCFLLP:/mnt/c/Users/Tushar Mangal/Documents/Containerization and DevOps/Lab/Lab-12$ kubectl delete pod wordpress-848c87c44f-7mzmk
pod "wordpress-848c87c44f-7mzmk" deleted from default namespace
tushar_mangal@LAPTOP-1DDBCFLLP:/mnt/c/Users/Tushar Mangal/Documents/Containerization and DevOps/Lab/Lab-12$ kubectl get pods
NAME                                READY    STATUS    RESTARTS   AGE
wordpress-848c87c44f-58fkg          1/1      Running   0           16s
wordpress-848c87c44f-djdnp          1/1      Running   0           3m4s
wordpress-848c87c44f-knb27          1/1      Running   0           55s
wordpress-848c87c44f-srbmb          1/1      Running   0           3m4s
```

69

20.1.0.6 When to Use Which Tool

Tool	Best for
k3d	Quick learning on your laptop
Minikube	Single-node cluster testing
kubeadm	Real, production-style cluster

20.1.0.7 Summary of Commands (Cheat Sheet)

Goal	Command
Apply a YAML file	<code>kubectl apply -f file.yaml</code>
See all pods	<code>kubectl get pods</code>
See all services	<code>kubectl get svc</code>
Scale a deployment	<code>kubectl scale deployment <name> --replicas=N</code>
Delete a pod	<code>kubectl delete pod <pod-name></code>
See all nodes	<code>kubectl get nodes</code>